12 Linux 页表体系结构-----虚拟内存与物理内存进行映射过程 2
12.1 虚拟内存与物理内存的映射原理 2
12.2 内核使用页表管理内存映射关系 4
12.2.1 用户态进程页表的创建和初始化过程 5
12.2.2 内核空间的页表创建和初始化过程 16
12.2.3 pgd_ctor 函数将内核空间的页表信息(init_mm.pgd)拷贝到进程空间的页表信息(mm->pgd)中 18
12.3 单级页表的不足 20
12.4 多级页表的演进 21
12.4.1 二级页表体系结构 22
12.4.2 32 位操作系统页表项 PTE 的结构 28
12.4.3 32 位操作系统页表项 PDE 的结构 32
12.4.4 四级页表结构 34
12.4.5 64 位系统下页表项 PTE 的结构 39
12.4.6 64 位系统下页目录项 PDE 的结构 40
12.5 CPU 的寻址过程 42

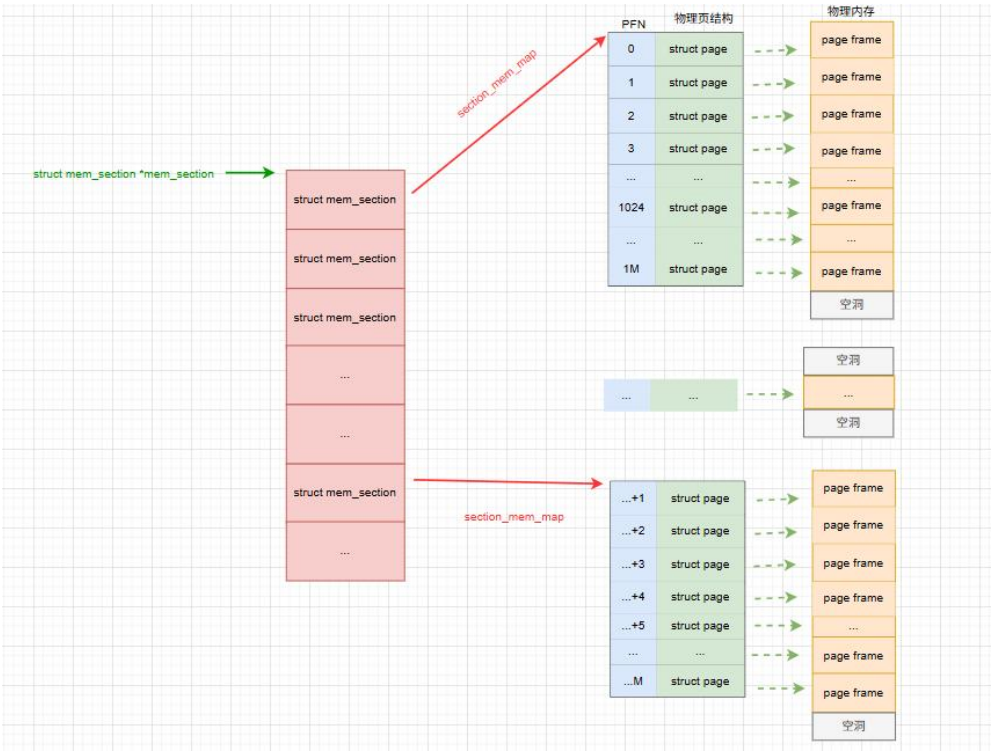
12 Linux 页表体系结构-----虚拟内存与物理内存进行映射过程

12.1 虚拟内存与物理内存的映射原理

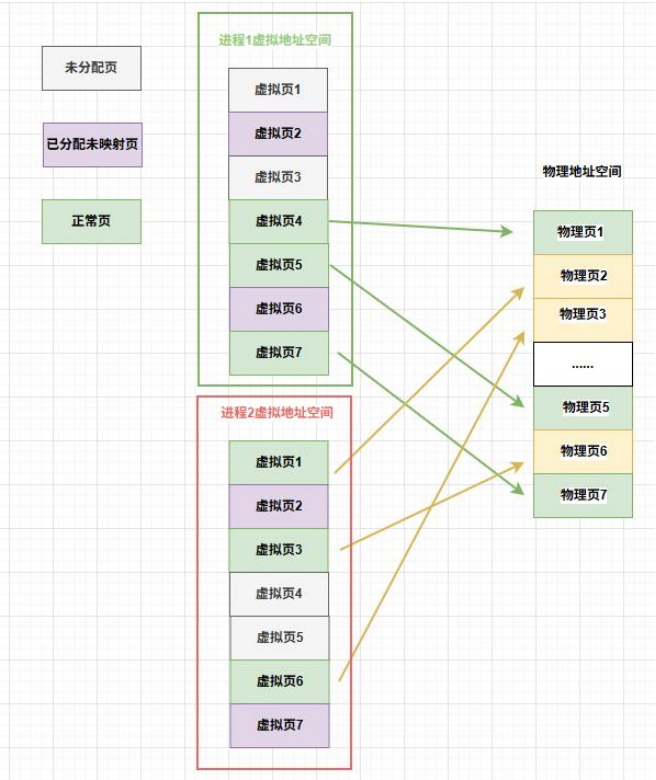
内核按照页为单位将整个物理内存空间划分为大小相同的的内存块，每个内存块大小为 4K，称为一个物理内存页。一页大小的内存块在内核中用 struct page 结构体来进行管理，struct page 中封装了每页内存块的状态信息，比如：组织结构，使用信息，统计信息，以及与其他内核结构的关联映射信息等。

内核会为每个物理内存页 page 进行统一编号。这个编号称之为 PFN（Page Frame Number），PFN 与 struct page 是一一对应的关系并且全局唯一。然后内核会将划分出来的这些一页一页的内存块统一组织在一个全局数组 mem_map 中管理。后续虚拟内存与物理内存的映射以及调度均是以页为单位进行的。

```
#ifdef CONFIG_SPARSEMEM_EXTREME
extern struct mem_section **mem_section;
#else
extern struct mem_section
mem_section[NR_SECTION_ROOTS][SECTIONS_PER_ROOT];
#endif
```



因为物理内存是以页为单位进行管理，而虚拟内存最终是要映射到物理内存上的，所以在虚拟内存空间中也有与之相对应的虚拟页这个概念，内存的映射是以页为单位进行的。



如上图所示，在内存映射的场景中，虚拟内存页的类型总共分为以下三种：

第一种就是图中灰色方框里标注的**未分配页面**，进程的虚拟内存空间非常庞大，远远的超过物理内存空间，但这并不意味着进程可以直接随意使用虚拟内存，事实上进程对虚拟内存的使用也是需要向内核申请的。进程虚拟内存空间中的虚拟内存页在未被进程申请之前的状态就是未分配页面。

第二种就是图中紫色方框里标注的**已分配未映射页面**，我们在进程中可以通过动态链接库 glic 中的 malloc 接口或者直接通过系统调用 mmap 向内核申请虚拟内存，申请到的虚拟内存页此时就变为了已分配的页面。但此时的虚拟内存页只是虚拟内存，其背后并没有与物理内存映射起来，所以称为已分配未映射页面。

第三种是图中绿色方框里标注的**正常页面**，当进程开始读写这些已分配未映射的虚拟内存页时，在 CPU 中用于地址翻译的硬件 MMU (负责将虚拟内存地址翻译为物理内存地址)会产生一个缺页中断，随后内核会为其分配相应的物理内存页面，并将虚拟内存页与物理内存页映射起来。此时这些已分配未映射的虚拟内存页就变为了正常页面。从此以后，进程就可以正常读写这些虚拟内存页了。

另外，从图中我们可以看出：

1. 每个进程独占全部的虚拟内存空间，比如上图中，进程 1 的虚拟内存空间（蓝色部分）和进程 2 的虚拟内存空间（黄色部分）它们都拥有属于各自的虚拟内存页 1 到虚拟内存页 7 这段范围的虚拟内存。也就是说进程 1 和进程 2 看到的虚拟内存空间地址范围都是一样的。

2. 每个进程的虚拟内存空间都相互隔离，互不干扰，进程可以在属于自己的虚拟内存空间里随意折腾。比如上图中，进程 1 里的虚拟内存页 1 是一个未分配页面，而进程 2 里的虚拟内存页 1 却是一个正常页面，被内核映射到物理内存页 2 中。也就是说虽然每个进程拥有的虚拟内存地址空间范围是一样的，但是各自虚拟内存空间中的虚拟页可能映射的物理页不一样，使用的方式和用途也不一样。

3. 进程所看到的连续虚拟内存，在物理内存中有可能是不连续的，比如上图中，进程 1 里的虚拟页 4 和 虚拟页 5，它们在进程 1 的虚拟内存空间中是连续的，但是它们背后映射的物理内存页却是不连续的。虚拟内存页 4 被映射到了物理内存页 1 中，虚拟内存页 5 被映射到了物理内存页 5 中。物理内存空间中蓝色部分是进程 1 正在使用的内存（物理页 1，物理页 5，物理页 7），黄色部分是进程 2 正在使用的内存（物理页 2，物理页 3，物理页 6）。这些复杂且琐碎的内存映射细节统统由内存管理子系统进行管理，从而极大的解放了程序员的心智负担。

进程的虚拟内存空间和物理内存空间通过页表建立映射关系。页表除了管理虚拟内存与物理内存之间的映射关系之外，还会有一些访问权限的管理，来控制进程对物理内存的访问权限。

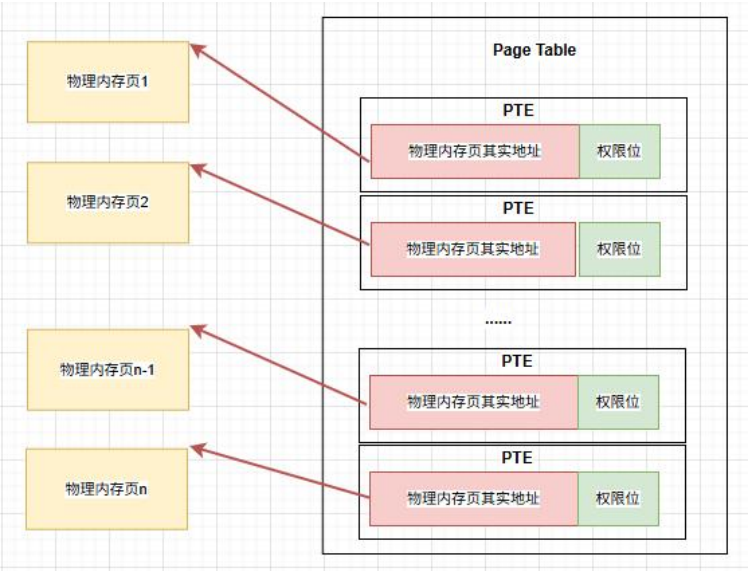
由于进程是独占虚拟内存空间的，而且不同进程之间的虚拟内存空间是相互隔离的，所以每个进程也都会有属于自己的页表，来专门管理各自虚拟内存空间中的映射关系以及各自访问物理内存的权限。

12.2 内核使用页表管理内存映射关系

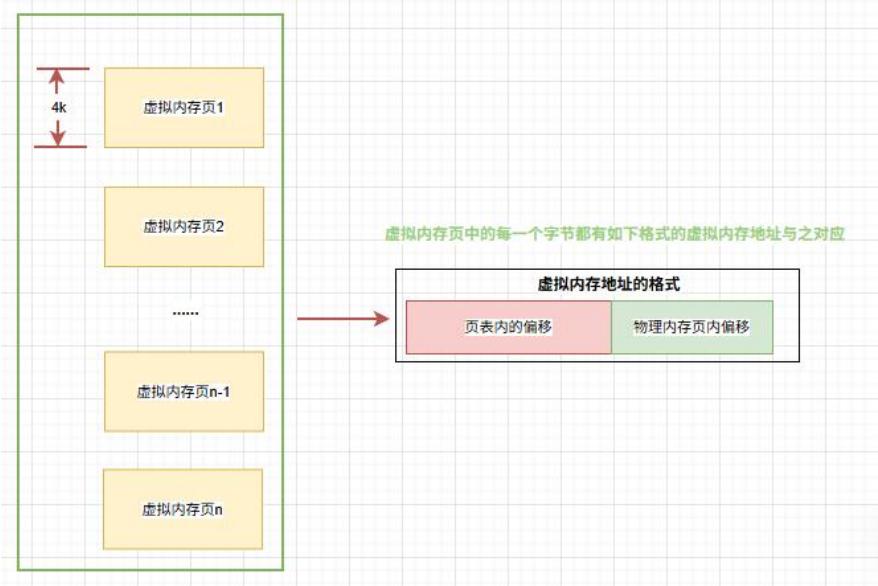
内核对物理内存的管理是按照页为基本单位进行的，进程运行起来所需要的数据也是存储在一个一个的物理页中，既然物理内存页可以存储进程的普通数据，那么它也一定可以存储进程虚拟内存与物理内存之间的映射关系。

事实上，内核会从物理内存空间中拿出一个物理内存页来专门存储进程里的这些内存映射关系，而这种物理内存页就称之为**页表**，可以看出**页表的本质其实就是一个物理内存页**。内核会在页表中划分出来一个个大小相等的小内存块，这些小内存块我们称之为页表项 PTE（Page Table Entry），正是这个 PTE 保存了进程虚拟内存空间中的虚拟页与物理内存页的映射关系，以及控制物理内存访问的相关权限位。在 linux 内核 32 位系统中页表中的 PTE 占用 4 个字节，64 位系统中页表的 PTE 占用 8 个字节。

因为内存映射的粒度是按照页为单位进行的，所以进程虚拟内存空间中的每个虚拟页在页表中都会有一个 PTE 与之对应，而虚拟页背后映射的物理内存页的起始地址就保存在 PTE 中。



进程虚拟内存空间中的每一个字节都有一个虚拟内存地址来表示，格式为：页表内偏移 + 物理内存页内偏移。其中，虚拟内存地址格式中的 页表内偏移 专门用来定位虚拟内存页在页表中的 PTE ，因为页表本质其实还是一个物理内存页，而一个物理内存页里边的内存肯定都是连续的，每个 PTE 的尺寸又是相同的，所以我们可以把页表看做一个数组，PTE 看做数组里的元素，在一个数组里定位元素，我们直接通过元素的索引 index 就可以定位了。这个索引 index 就是 页表内偏移 。



因此，给定一个虚拟内存地址，内核会先从这个虚拟内存地址中提取出 页表内偏移 ，然后根据 页表起始地址 + 页表内偏移 * sizeof(PTE) 就能获取到该虚拟内存地址所在虚拟页在页表中对应的 PTE 。

12.2.1 用户态进程页表的创建和初始化过程

页表起始地址保存在 struct mm_struct 结构中的 pgd 属性中。内核中用 struct mm_struct 结构来描述进程的虚拟内存空间，每个进程都有自己独立的虚拟内存空间，而进程的虚拟内存到物理内存的映射也是独立的，为了保证每个进程里内存映射的独立进行，每个进程都会有独立的页表，而页表的起始地址就存放在 struct mm_struct 结构中的 pgd 属性中。注意，mm_struct->pgd 存放的是进程的顶级页表的起始地址。

```
struct mm_struct {
    ......
    pgd_t * pgd;
    ......

}
```

pgd 的值是在进程被创建时被设置的，使用 fork 系统调用创建进程的时候，内核在 kernel_clone 函数中会通过 copy_process 将父进程的所有资源拷贝到子进程中，这其中也包括父进程的虚拟内存空间。

```
    pid_t kernel_clone(struct kernel_clone_args *args)
{
    .......省略......
    struct pid *pid;
    struct task_struct *p;
    .......省略......
    // 拷贝父进程的所有资源
    p = copy_process(NULL, trace, NUMA_NO_NODE, args);
    add_latent_entropy();
    .......省略......
}
```

copy_process 函数开始拷贝父进程中的所有资源到子进程中：

```
__latent_entropy struct task_struct *copy_process(
                    struct pid *pid,
                    int trace,
                    int node,
                    struct kernel_clone_args *args)
{
    int pidfd = -1, retval;
    struct task_struct *p;
    .......省略......
     // 为进程创建 task_struct 结构
    p = dup_task_struct(current, node);
    if (!p)
        goto fork_out;

     .......初始化子进程......
```

```
    // 拷贝父进程的虚拟内存空间以及页表
    retval = copy_mm(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_signal;

    ......... 拷贝父进程的其他资源 .........

    stackleak_task_init(p);
    if (pid != &init_struct_pid) {
     // 分配 pid
        pid = alloc_pid(p->nsproxy->pid_ns_for_children, args->set_tid,
                args->set_tid_size);
        if (IS_ERR(pid)) {
            retval = PTR_ERR(pid);
            goto bad_fork_cleanup_thread;
        }
    }
     .......省略......
}
```

copy_mm 函数负责处理子进程虚拟内存空间的初始化工作，然后调用 dup_mm 函数，最终在 dup_mm 函数中将父进程虚拟内存空间的所有内容包括父进程的相关页表全部拷贝到子进程中，其中就包括了为子进程分配顶级页表起始地址 pgd。

```
static int copy_mm(unsigned long clone_flags, struct task_struct *tsk)
{
    struct mm_struct *mm, *oldmm;

    ......初始化统计字段......
    ......清除 mm 相关指针......
    //判断是否为内核线程
    //current 是当前执行的进程(即父进程).
    //如果 current->mm == NULL，说明当前是内核线程（无用户地址空间）
    //此时子任务若也是内核线程（比如通过 kthread_create 创建），
    //则不需要 mm_struct，直接返回 0。
    //内核线程没有 mm_struct，它们共享内核地址空间
    oldmm = current->mm;
    if (!oldmm)
        return 0;
      //根据 CLONE_VM 决定共享 or 复制
      //CLONE_VM 被设置（如 vfork()、pthread_create()）
      //子进程与父进程共享同一个 mm_struct（即地址空间）。
    if (clone_flags & CLONE_VM) {
    //增加引用计数，防止父进程退出时提前释放 mm
        mmget(oldmm);
        mm = oldmm;
```

```
    } else {
    //调用 dup_mm(tsk, current->mm) 深度复制父进程的地址空间，
    //包括页表、VMA（虚拟内存区域）、内存描述符等。
    //dup_mm 函数会分配新的 mm_struct 结构体，
    //并复制原 mm_struct 的相关内容，包括内存映射等信息。
    //使用写时复制（COW）机制：物理页初始共享，写入时才复制。
        mm = dup_mm(tsk, current->mm);
        if (!mm)
          //分配失败（如无法申请 mm_struct 或 VMA 结构），返回 -ENOMEM
            return -ENOMEM;
    }
    //对于普通进程 active_mm 和 mm 相同。
    //对于内核线程：mm = NULL, active_mm 可能指向某个用户进程的 mm(用于访问用户空间，如 copy_from_user)
    tsk->mm = mm;
    tsk->active_mm = mm;
    sched_mm_cid_fork(tsk);
    return 0;
}
```

dup_mm 的作用是复制一个已有的内存管理结构体(mm_struct)。为新任务 task_struct ，创建一个与给定的 oldmm 结构内容相同的新的 mm_struct 结构。如果复制过程中出现错误，函数将返回 NULL，否则返回新复制的 mm_struct 结构体指针。

```
/**
 * dup_mm() - duplicates an existing mm structure
 * @tsk: the task_struct with which the new mm will be associated.
 * @oldmm: the mm to duplicate.
 *
 * Allocates a new mm structure and duplicates the provided @oldmm structure
 * content into it.
 *
 * Return: the duplicated mm or NULL on failure.
 */
static struct mm_struct *dup_mm(struct task_struct *tsk,
                struct mm_struct *oldmm)
{
    //子进程虚拟内存空间，此时还是空的
    struct mm_struct *mm;
    int err;
    // 为子进程申请 mm_struct 结构
    mm = allocate_mm();
    if (!mm)
        goto fail_nomem;
    //浅拷贝整个 mm_struct
    //使用 memcpy 函数将 oldmm 的内容复制到新分配的 mm 结构体中。
```

```
    //直接复制整个 mm_struct 的内容，包括其中的各种成员变量和设置。
    memcpy(mm, oldmm, sizeof(*mm));
     // 为子进程分配顶级页表起始地址并赋值给 mm_struct->pgd
    //初始化新的 mm_struct
    if (!mm_init(mm, tsk, mm->user_ns))
        goto fail_nomem;
    //复制内存映射。
    // 复制 oldmm 的内存映射到新的 mm 结构体中。
    err = dup_mmap(mm, oldmm);
    if (err)
        goto free_pt;
    //设置高水位标记
    //获取新 mm 结构体当前的驻留集大小（RSS）并设置为高水位标记 hiwater_rss
    mm->hiwater_rss = get_mm_rss(mm);
    //将新 mm 结构体的总虚拟内存大小 total_vm 设置为高水位标记 hiwater_vm。
    mm->hiwater_vm = mm->total_vm;
    if (mm->binfmt && !try_module_get(mm->binfmt->module))
        goto free_pt;
    //返回新复制并初始化好的 mm_struct 结构体指针 mm
    return mm;
free_pt:
    /* don't put binfmt in mmput, we haven't got module yet */
    mm->binfmt = NULL;
    mm_init_owner(mm, NULL);
    mmput(mm);
fail_nomem:
    return NULL;
}
```

内核会在 mm_init 函数中调用 mm_alloc_pgd，并在 mm_alloc_pgd 函数中通过调用 pgd_alloc 为子进程分配其独立的顶级页表起始地址，并赋值给子进程 struct mm_struct 结构中的 pgd 属性。

```
static struct mm_struct *mm_init(struct mm_struct *mm, struct task_struct
*p,
    struct user_namespace *user_ns)
{
   //Memory Tree (mt) 初始化 —— 新版内存管理基础设施
    //mm_mt 是 Memory Tree
    mt_init_flags(&mm->mm_mt, MM_MT_FLAGS);
    mt_set_external_lock(&mm->mm_mt, &mm->mmap_lock);
    //引用计数初始化
    atomic_set(&mm->mm_users, 1); // 用户可见引用数。mm_users：被多少
                                  //task_struct 持有（线程共享时 >1）
    //mm_count：被多少地方间接持有（如 exe_file, vma->vm_file 可能隐式持有）
    atomic_set(&mm->mm_count, 1);
```

```
    //写保护序列号
    seqcount_init(&mm->write_protect_seq);
    //锁初始化
    mmap_init_lock(mm);
    //链表与统计初始化
    INIT_LIST_HEAD(&mm->mmlist);
#ifdef CONFIG_PER_VMA_LOCK
    mm->mm_lock_seq = 0;
#endif
    mm_pgtables_bytes_init(mm);
    mm->map_count = 0;
    mm->locked_vm = 0;
    atomic64_set(&mm->pinned_vm, 0);
    memset(&mm->rss_stat, 0, sizeof(mm->rss_stat));
    spin_lock_init(&mm->page_table_lock);
    spin_lock_init(&mm->arg_lock);
    //CPU Mask / AIO / Owner 初始化
    mm_init_cpumask(mm);// 分配 cpumask 用于 NUMA 亲和性
    mm_init_aio(mm);// 初始化异步 I/O 上下文（aio_ring）
    mm_init_owner(mm, p); // 设置 mm->owner = p（当前 task）
    mm_pasid_init(mm);
    ......省略......
    //mmf_init_flags(): 清除 不可继承标志（如 MMF_OOM_SKIP, MMF_UNSTABLE）
    //, 保留 MMF_DUMPABLE 等。
    //VM_INIT_DEF_MASK: 只继承 VM_READ|VM_WRITE|VM_EXEC|VM_MAY*
    //等初始标志，过滤运行时标志（如 VM_LOCKED）。
    //确保子进程不会意外继承父进程的调试/特殊状态
    if (current->mm) {
        mm->flags = mmf_init_flags(current->mm->flags);
        mm->def_flags = current->mm->def_flags & VM_INIT_DEF_MASK;
    } else {
        mm->flags = default_dump_filter;
        mm->def_flags = 0;
    }
     // 为子进程分配顶级页表起始地址 pgd
    if (mm_alloc_pgd(mm))
        goto fail_nopgd;
    if (init_new_context(p, mm))
        goto fail_nocontext;
    if (mm_alloc_cid(mm))
        goto fail_cid;
    if (percpu_counter_init_many(mm->rss_stat, 0, GFP_KERNEL_ACCOUNT,
                    NR_MM_COUNTERS))
        goto fail_pcpu;
```

```
    mm->user_ns = get_user_ns(user_ns);
    lru_gen_init_mm(mm);
    return mm;
//错误处理路径详解（逆向清理顺序）
fail_pcpu:
    // 先销毁 CID（依赖 percpu）
    mm_destroy_cid(mm);
fail_cid:
   // 销毁 ASID/架构上下文
    destroy_context(mm);
fail_nocontext:
     // 释放 PGD（最底层资源）
    mm_free_pgd(mm);
fail_nopgd:
    // 最终释放 mm_struct 本身
    free_mm(mm);
    return NULL;
}
```

mm_alloc_pg 函数核心职责:

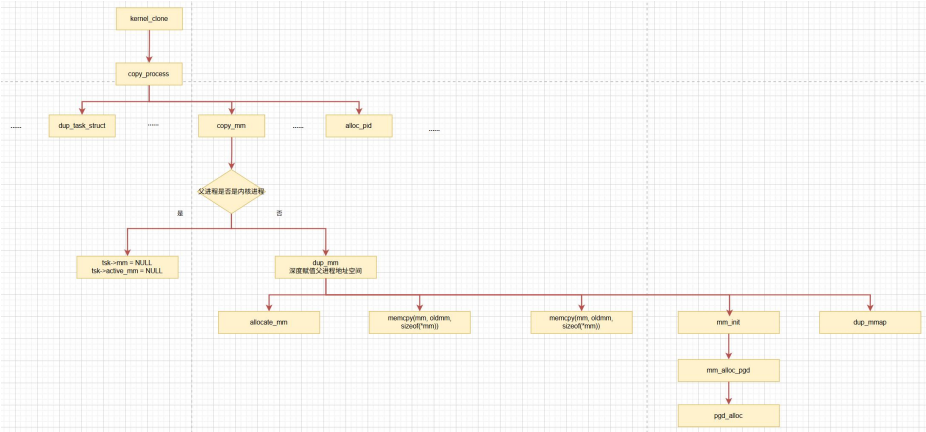
1、为 mm 分配用户空间页表根（PGD）

2、原子性地将指针写入 mm->pgd

3、失败时返回 -ENOMEM，供上层统一清理

```
static inline int mm_alloc_pgd(struct mm_struct *mm)
{
    // 内核为子进程分配好其顶级页表起始地址之后
    // 赋值给子进程 mm_struct 结构中的 pgd 属性
    mm->pgd = pgd_alloc(mm);
    if (unlikely(!mm->pgd))
        return -ENOMEM;
    return 0;
}
```

上述代码描述的进程创建的整体流程：



到现在为止，一个进程就算是被完整的创建出来，它拥有了自己独立的页表（页表内容和父进程一模一样），同时也拥有了属于自己的顶级页表起始地址 pgd，但是需要特别注意一点,进程的 struct mm_struct 结构中的这 pgd 现在还只是顶级页表的虚拟内存地址，此时未被 CPU 直接使用。

当这个进程被调度到某个 CPU 之上时，内核就会调用 context_switch 来对进程上下文进行切换，切换的内容主要包括：

1、进程虚拟内存空间的切换。

2、寄存器以及进程栈的切换。

```
/*
 * context_switch - switch to the new MM and the new thread's register state.
 */
static __always_inline struct rq *
context_switch(struct rq *rq, struct task_struct *prev,
          struct task_struct *next, struct rq_flags *rf)
{
    prepare_task_switch(rq, prev, next);
    arch_start_context_switch(prev);
    if (!next->mm) {                                // to kernel
         ......内核线程切换......
    /*
    *进入延迟 TLB（Translation Look - aside Buffer）模式。
    *TLB 用于加速虚拟地址到物理地址的转换，
    *延迟 TLB 模式允许内核线程共享前一个任务（prev）的 TLB 信息，
    *减少不必要的 TLB 刷新，从而提高上下文切换效率。
    */
             enter_lazy_tlb(prev->active_mm, next);
             //内核线程复用前一个任务的 active_mm
             //实现内存管理上下文的共享
              next->active_mm = prev->active_mm;
              if (prev->mm)                          // from use
               //如果 prev 任务是从用户态切换过来的
               //在延迟 TLB 模式下获取对 prev 任务活动内存管理结构体的引用，
               //保证在共享内存管理上下文时内存访问的正确性。
                    mmgrab_lazy_tlb(prev->active_mm);
              else
                  //如果 prev 任务本身就是内核线程（没有 mm 结构体），
                   //将 prev->active_mm 设置为 NULL，
                   //表示 prev 不再有活动的内存管理结构体。
                   prev->active_mm = NULL;
    } else {                                        // to user
       ......用户线程切换......
```

```
        membarrier_switch_mm(rq, prev->active_mm, next->mm);
       // 切换进程虚拟内存空间
       /*
       *在关闭中断的情况下切换内存管理上下文。
       *关闭中断保证了内存管理切换操作的原子性，
       *避免在切换过程中被中断干扰，确保内存管理数据结构的完整性和一致性。
       */
       switch_mm_irqs_off(prev->active_mm, next->mm, next);
     /*
     *更新与最近最少使用（LRU）算法相关的统计信息。
     *LRU 算法用于内存页面置换，通过更新这些统计信息，
     *系统能更好地管理内存，决定哪些页面可以被置换以释放内存空间。
     */
       lru_gen_use_mm(next->mm);
       if (!prev->mm) {                        // from kernel
           /* will mmdrop_lazy_tlb() in finish_task_switch(). */
           rq->prev_mm = prev->active_mm;
           prev->active_mm = NULL;
       }
   }
   //切换内存上下文标识符（CID）。
   switch_mm_cid(rq, prev, next);
   prepare_lock_switch(rq, next, rf);
   //寄存器和栈切换。
   /*
   *保存当前任务（prev）的寄存器状态，
   *加载下一个任务（next）的寄存器状态，
   *并切换栈指针，使得 CPU 开始执行 next 任务的代码。
   */
   switch_to(prev, next, prev);
   barrier();
   return finish_task_switch(prev);
}
```

注：为什么内核线程需要 active_mm？

为了让内核线程能够访问内存，Linux 内核引入了 active_mm 机制。当内核线程被调度运行时，它会将 active_mm 指针指向当前正在运行的进程（可能是用户态进程，也可能是其他内核线程）的 mm_struct。这样，内核线程就通过 active_mm 间接获取了内存管理上下文。

switch_mm_irqs_off 函数，它主要负责对进程虚拟内存空间进行切换，其中就包括了调用 load_new_mm_cr3 函数将进程顶级页表起始地址 mm_struct-> pgd 中的虚拟内存地址通过 __sme_pa 宏 转换为物理内存地址，并将 pgd 的物理内存地址加载到 cr3 寄存器中。

```
 void switch_mm_irqs_off(struct mm_struct *prev, struct mm_struct *next,
          struct task_struct *tsk)
{
```

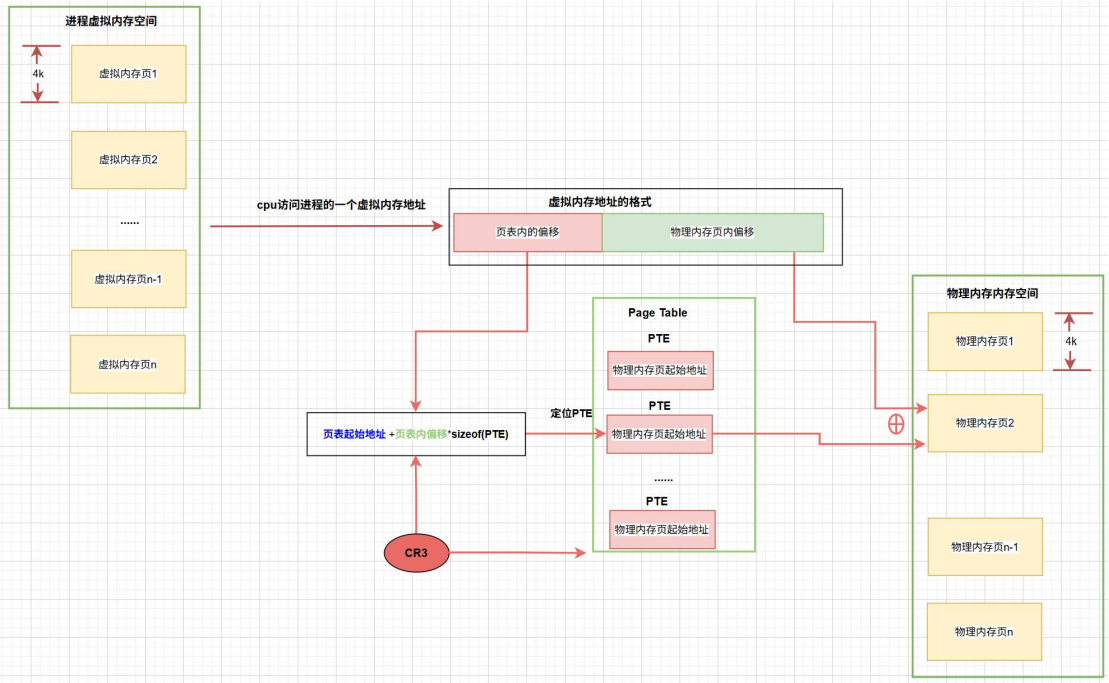
```
    ......省略......
    //执行页表切换与 TLB 状态同步。
    //根据是否需要刷新 TLB（need_flush）分两条路径执行
    if (need_flush) {
    //TLB 内容已过期，必须刷新
    //将当前 CPU 的 new_asid 上下文绑定到目标 mm 的全局唯一 ID
        this_cpu_write(cpu_tlbstate.ctxs[new_asid].ctx_id,
next->context.ctx_id);
    //更新该 ASID 对应的 TLB 代际版本号
        this_cpu_write(cpu_tlbstate.ctxs[new_asid].tlb_gen,
next_tlb_gen);
    //构建并加载新 CR3 寄存器值
        load_new_mm_cr3(next->pgd, new_asid, new_lam, true);
        //向 ftrace 提供事件，用于性能分析。
        trace_tlb_flush(TLB_FLUSH_ON_TASK_SWITCH, TLB_FLUSH_ALL);
    } else {
    //TLB 内容仍有效，仅切换地址空间
    //直接写 CR3（复用现有 TLB entry），不刷新 TLB。
        load_new_mm_cr3(next->pgd, new_asid, new_lam, false);
      //pages=0 表示无实际 flush（仅地址空间切换）
        trace_tlb_flush(TLB_FLUSH_ON_TASK_SWITCH, 0);
    }
    //确保 CR3 写入完成后再更新 loaded_mm
    barrier();
    this_cpu_write(cpu_tlbstate.loaded_mm, next);
    this_cpu_write(cpu_tlbstate.loaded_mm_asid, new_asid);
    if (next != real_prev) {
        cr4_update_pce_mm(next);
        switch_ldt(real_prev, next);
    }
}
```

load_new_mm_cr3() 是 x86_64 内核中构建并加载新 CR3 寄存器值的核心函数，cr3 寄存器中存放的是当前进程顶级页表 pgd 的物理内存地址，不能是虚拟内存地址。

```
static void load_new_mm_cr3(pgd_t *pgdir, u16 new_asid, unsigned long lam,
                bool need_flush)
{
  /*pgdir: 新 mm 的 PGD 物理地址
    *逻辑标志，表示“是否需显式刷新 TLB”
    *need_flush 不是直接控制硬件 flush，
    *而是指导软件是否需调用 invalidate_user_asid()
    */
    unsigned long new_mm_cr3;
    if (need_flush) {
    //清除 该 ASID 在当前 CPU 的 TLB 缓存条目（仅限 user space）
```

```
        invalidate_user_asid(new_asid);
      //构建完整的 CR3 值（物理地址 + ASID + LAM）
        new_mm_cr3 = build_cr3(pgdir, new_asid, lam);
    } else {
    //不需要刷新 TLB ，仅需更新 CR3 寄存器
        new_mm_cr3 = build_cr3_noflush(pgdir, new_asid, lam);
    }
    //将页表基址写入 CPU 的 CR3 寄存器
    write_cr3(new_mm_cr3);
}
```

进程的上下文在内核中完成切换之后，现在 cr3 寄存器中保存的就是当前进程顶级页表的起始物理内存地址，当 CPU 通过下图所示的虚拟内存地址访问进程的虚拟内存时，CPU 首先会从 cr3 寄存器中获取到当前进程的顶级页表起始地址，然后从虚拟内存地址中提取出虚拟内存页对应 PTE 在页表内的偏移，通过 页表起始地址 + 页表内偏移 * sizeof(PTE) 这个公式定位到虚拟内存页在页表中所对应的 PTE。



虚拟内存页背后所映射的物理内存页的起始地址就保存在该 PTE 中，随后 CPU 继续从上图所示的虚拟内存地址中提取后半部分——物理内存页内偏移，并通过 物理内存页起始地址 + 物理内存页内偏移 就定位到该物理内存页中一个具体的物理字节上。

当用户进程被 CPU 调度起来，访问进程虚拟内存的时候，上述的虚拟内存地址与物理内存地址转换的过程都是在用户态进行的，正常的内存访问无需进入内核态。除非 CPU 访问的虚拟内存页面类型是：

1、未分配页面。

2、已分配未映射页面。

3、已映射，但是由于内存紧张的原因，该虚拟内存页映射的物理内存页被置换到磁盘上了。

以上三种虚拟内存页有一个共同的特征就是它们背后的物理内存页均不在内存中，要么是没有映射，要么是被置换到磁盘上。当 CPU 访问这些虚拟内存页面的时候，就会产生缺

页中断，随后进入内核态为其分配物理内存页面，填充物理内存页面中的内容，最后在页表中建立映射关系。之后的内存访问均是在用户态中进行。

12.2.2 内核空间的页表创建和初始化过程

进程的整个虚拟内存空间分为两个部分：

1、用户态虚拟内存空间

2、内核态虚拟内存空间。

以 64 位操作系统为例：

64 位机器上linux/用户虚拟内存空间划分

CPU 无论是在用户态还是在内核态，访问的均是虚拟内存地址，不管是用户空间的虚拟内存地址还是内核空间的虚拟内存地址最终都是要与物理内存进行映射的，虚拟内存与物理内存的映射关系通过页表管理。所以页表也分为两个部分：

1、进程用户态页表：主要负责管理进程用户态虚拟内存空间到物理内存的映射关系。

2、内核态页表：主要负责管理内核态虚拟内存空间到物理内存的映射关系，这一部分主要供内核使用。

和进程用户态虚拟内存空间一样，内核态虚拟内存空间也有一个 struct mm_struct 结构来描述：

```
struct mm_struct init_mm = {
    .mm_mt      = MTREE_INIT_EXT(mm_mt, MM_MT_FLAGS, init_mm.mmap_lock),
    .pgd        = swapper_pg_dir,
    .mm_users   = ATOMIC_INIT(2),
    .mm_count   = ATOMIC_INIT(1),
    .write_protect_seq = SEQCNT_ZERO(init_mm.write_protect_seq),
    MMAP_LOCK_INITIALIZER(init_mm)
    .page_table_lock =  __SPIN_LOCK_UNLOCKED(init_mm.page_table_lock),
    .arg_lock   =  __SPIN_LOCK_UNLOCKED(init_mm.arg_lock),
    .mmlist     = LIST_HEAD_INIT(init_mm.mmlist),
#ifdef CONFIG_PER_VMA_LOCK
    .mm_lock_seq    = 0,
#endif
    .user_ns    = &init_user_ns,
    .cpu_bitmap = CPU_BITS_NONE,
    INIT_MM_CONTEXT(init_mm)
};
```

内核的页表在系统初始化的时候被一段汇编代码 arch\x86\kernel\head_64.S 所创建。后续内核虚拟内存空间的创建以及内核页表的初始化工作是在系统启动函数 start_kernel 中调用 setup_arch 完成。

可以看出，内核空间的顶级页表起始地址 pgd 叫做 swapper_pg_dir，定义在文件 arch/x86/include/asm/pgtable_64.h 中：

```
#define swapper_pg_dir init_top_pgt
```

内核的页表在系统初始化的时候被一段汇编代码 arch\x86\kernel\head_64.S 所创建。后续内核虚拟内存空间的创建以及内核页表的初始化工作是在系统启动函数 start_kernel 中调用 setup_arch 完成。

```
asmlinkage __visible __init __no_sanitize_address __noreturn
__no_stack_protector
void start_kernel(void)
{
    ...... 省略......
     // 创建内核虚拟内存空间，初始化内核页表
    setup_arch(&command_line);
    ...... 省略......
}
```

```
void __init setup_arch(char **cmdline_p)
{
#ifdef CONFIG_X86_32
    memcpy(&boot_cpu_data, &new_cpu_data, sizeof(new_cpu_data));
    // 初始化内核页表
    clone_pgd_range(swapper_pg_dir     + KERNEL_PGD_BOUNDARY,
            initial_page_table + KERNEL_PGD_BOUNDARY,
            KERNEL_PGD_PTRS);
     // 将内核顶级页表起始地址转换为物理地址，并加载到 cr3 寄存器中
    load_cr3(swapper_pg_dir);
      // 刷新 TLB 页表缓存
    __flush_tlb_all();
#else
    printk(KERN_INFO "Command line: %s\n", boot_command_line);
    boot_cpu_data.x86_phys_bits = MAX_PHYSMEM_BITS;
#endif
......省略......
}
```

12.2.3 pgd_ctor 函数将内核空间的页表信息(init_mm.pgd)拷贝到进程空间的页表信息(mm->pgd)中

无论是进程页表还是内核页表，在被 CPU 访问之前都必须先加载到 cr3 寄存器中。内核页表已经被创建和初始化如上。但是对于处于内核态的进程以及内核线程来说并不能直接访问这个内核页表，它们只能访问内核页表的 copy 副本，进程的页表分为两个部分，一个是进程用户态页表，另一个就是内核页表的 copy 部分。

如前面提到的 fork 系统调用在创建子进程的流程，会拷贝父进程的所有资源，当拷贝父进程的虚拟内存空间的时候，内核会通过 pgd_alloc 函数为子进程创建顶级页表 pgd，其实还有一项重要的工作，就是在 pgd_alloc 函数中还会调用 pgd_ctor， pgd_ctor 函数的主要工作就是将内核页表拷贝到进程页表中。

```
static inline int mm_alloc_pgd(struct mm_struct *mm)
{
    // 内核为子进程分配好顶级页表起始地址之后
    // 赋值给子进程 mm_struct 结构中的 pgd 属性
    mm->pgd = pgd_alloc(mm);
    if (unlikely(!mm->pgd))
        return -ENOMEM;
    return 0;
}

pgd_t *pgd_alloc(struct mm_struct *mm)
{
  pgd_t *pgd;
  ......省略......
  // 为子进程分配顶级页表
    pgd = _pgd_alloc();
    if (pgd == NULL)
        goto out;
  mm->pgd = pgd;
  ......省略......

  // 拷贝内核页表到子进程中
  pgd_ctor(mm, pgd);
  ......省略......
}


static void pgd_ctor(struct mm_struct *mm, pgd_t *pgd)
{
    if (CONFIG_PGTABLE_LEVELS == 2 ||
        (CONFIG_PGTABLE_LEVELS == 3 && SHARED_KERNEL_PMD) ||
```

```
        CONFIG_PGTABLE_LEVELS >= 4) {
//从 swapper_pg_dir（通常是内核启动时使用的页全局目录）中克隆一部分页全局目录
项到新的 pgd 中。
//新进程的 pgd 中与内核相关的部分页表映射信息会与 swapper_pg_dir 中的相应部分
//相同，保证了新进程能够正确访问内核空间。
        clone_pgd_range(pgd + KERNEL_PGD_BOUNDARY,
                swapper_pg_dir + KERNEL_PGD_BOUNDARY,
                KERNEL_PGD_PTRS);
    }
    if (!SHARED_KERNEL_PMD) {
    //将 pgd 与给定的 mm_struct（mm）关联起来。在 Linux 内核内存管理中，
    //这种关联是必要的，它使得 mm_struct 能够通过 pgd
    //来管理进程的虚拟地址空间映射。
        pgd_set_mm(pgd, mm);
//将 pgd 添加到一个特定的列表中。这个列表的作用是用于同步内核映射的更新。当内核
//映射发生变化时，通过这个列表可以方便地遍历所有相关的 pgd，并进行相应的更新操
//作，确保所有进程的内核映射信息保持一致。
        pgd_list_add(pgd);
    }
}
```

当进程通过系统调用切入到内核态之后，就会使用内核页表的这部分 copy 副本，来访问内核空间虚拟内存映射的物理内存。当进程页表中内核部分的拷贝副本与主内核页表不同步时，进程在内核态就会发生缺页中断，随后会同步主内核页表到进程页表中，这种情况又是延时拷贝在内核中的一处应用。

内核线程有一点和普通的进程不同，内核线程只能运行在内核态，而在内核态中，所有进程看到的虚拟内存空间全部都是一样的，所以对于内核线程来说并不需要为其单独的定义 mm_struct 结构来描述内核虚拟内存空间，内核线程的 struct task_struct 结构中的 mm 属性指向 null，内核线程之间调度是不涉及地址空间切换的，从而避免了无用的 TLB 缓存以及 CPU 高速缓存的刷新。

```
struct task_struct {
    ......省略......
    struct mm_struct        *mm;
    //核线程的 active_mm 指向前一个进程的地址空间
    // 普通进程的 active_mm 指向 null
    struct mm_struct        *active_mm;
     ......省略......
}
```

因为内核线程不会访问用户空间的内存，它仅仅只会访问内核空间的内存，所以直接复用上一个用户态进程页表的内核部分就可以避免为内核线程分配 mm_struct 和相关页表的开销，以及避免内核线程之间调度时地址空间的切换开销。

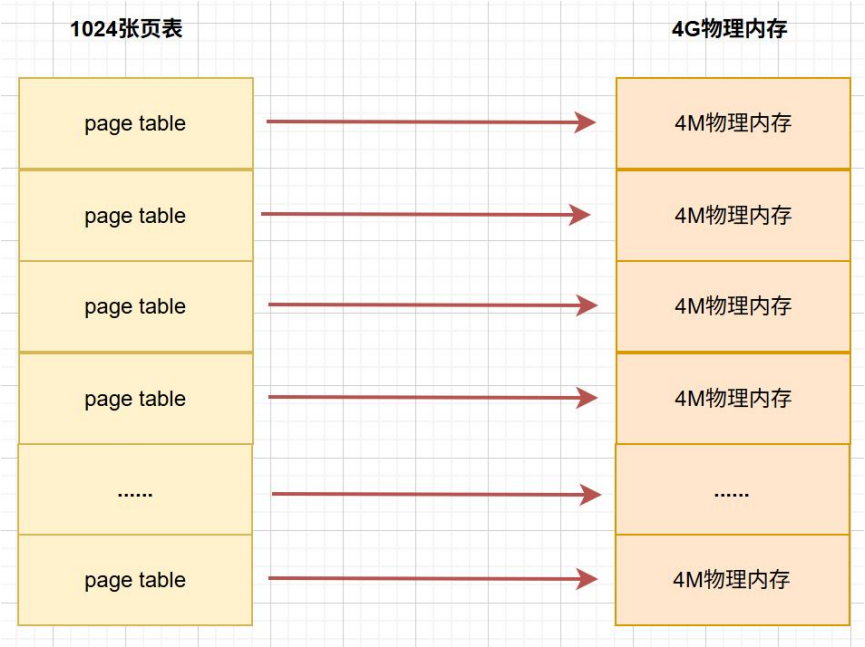
12.3 单级页表的不足

页表的本质其实就是一个物理内存页，一张页表 4K 大小，以 32 位系统为例，页表中的一个 PTE 占用 4B(32 位) 大小，所以一张页表可以容纳 1024 个 PTE。

在进程中虚拟内存与物理内存的映射是以页为单位的，进程虚拟内存空间中的一个虚拟内存页映射物理内存空间的一个物理内存页，这种映射关系以及访存权限都保存在 PTE 中，所以进程中的一个虚拟内存页对应页表中的一个 PTE，一个 PTE 能够映射 4K 的物理内存（一个物理内存页）。

一张页表里边可以容纳 1024 个 PTE，一个 PTE 可以映射 4K 的物理内存，那么一张页表就可以映射 1024 * 4K = 4M 大小的物理内存 ，而页表本质上是一个物理内存页（4K 大小），所以内核需要用额外的 4K 大小的物理内存去映射 4M 的物理内存。

如果系统中有 4G 的物理内存，一张页表能够映射 4M 大小的物理内存，而为了映射这 4G 的物理内存，需要 1024 张页表，一张页表占用 4K 物理内存，所以为了映射 4G 的物理内存，我们额外需要 4M 的物理内存（1024 张页表）来映射。



重要的是这 4M 物理内存（1024 张页表）还必须是连续的，因为页表是单级的，而页表相当于是 PTE 的数组，进程虚拟内存空间中的一个虚拟内存页对应一个 PTE，而 PTE 在页表这个数组中的索引 index 就保存在虚拟内存地址中，内核通过页表的起始地址加上这个索引 index 才能定位到虚拟内存页对应的 PTE，近而通过 PTE 定位到映射的物理内存页。

如果这 4M 物理内存（1024 张页表)不是连续的，就无法通过访问数组的方式定位 PTE 。而系统经过长时间运行之后，由于内存碎片的原因，是很难找到这么大一片连续的物理内存的。

需要注意的是，这 4M 的连续物理内存还只是一个进程所需要的，因为进程的虚拟内存空间都是独立的，页表也是独立的，一个进程就需要额外的 4M 连

续物理内存（1024 张页表）来支持进程内独立的内存映射关系。假如在系统中跑上 100 个进程，那总共就需要额外的 400M 连续的物理内存。这对于一个只有 4G 物理内存，单级页表的系统来说，无疑是巨大的开销和浪费。

如果进程一启动就立马会访问全部的 4G 物理内存，就需要在一开始就为进程分配 4M 的连续物理内存来存放页表，那这一点开销无论多么大都是必须的，不能省的，否则进程将无法运行。

由程序的局部性原理:包括时间局部性和空间局部性。

时间局部性是指如果程序中的某条指令一旦执行，则不久之后该指令可能再次被执行；如果某块数据被访问，则不久之后该数据可能再次被访问。

空间局部性是指一旦程序访问了某个存储单元，则不久之后，其附近的存储单元也将被访问。

可知，时间局部性和空间局部性。时间局部性是指如果程序中的某条指令一旦执行，则不久之后该指令可能再次被执行；如果某块数据被访问，则不久之后该数据可能再次被访问。空间局部性是指一旦程序访问了某个存储单元，则不久之后，其附近的存储单元也将被访问。

所以无论一个进程在实际运行过程中总共需要占用的内存资源有多大，根据程序局部性原理，在某一段时间内，进程真正需要的物理内存其实是很少的一部分，我们只需要为每个进程分配很少的物理内存就可以保证进程的正常执行运转。

既然在某一个特定的时刻，进程只需要很少的物理内存就可以正常运转，那么进程虚拟内存与物理内存之间的映射关系相应也会很少，根本就不需要 4M 的物理内存来保存映射关系。

完全可以在进程初始状态下，创建一个最小集的页表，当进程实际确实需要的时候，我们再来创建相应具体的页表，这又是延时分配思想在内核中的另一处应用。

12.4 多级页表的演进

几个重要的概念：

1、页表本质上还是一个物理内存页，只不过这个物理内存页比较特殊，里面存放的是 PTE，保存虚拟内存与物理内存的映射关系。既然它是一个普通的物理内存页，那么也会参与内核的调度，既会被内核 swap in 以及 swap out，也会被缓存在 CPU 高速缓存中加速访问。

2、在 32 位系统中，一个 PTE 占用 4 个字节，可以映射 4K 的物理内存，一张页表本身占用 4K 的物理内存(1024 个 PTE)，可以映射 4M 的物理内存。

3、定位虚拟内存页在进程页表中对应的 PTE 是通过数组的访问方式进行的，虚拟内存地址中包含了其对应的 PTE 在页表中的偏移（页表数组中的 index），所以就要求每一级页表都必须是连续的，比如单级页表中，这 1024 张页表必须是连续的物理内存（4M 大小）。

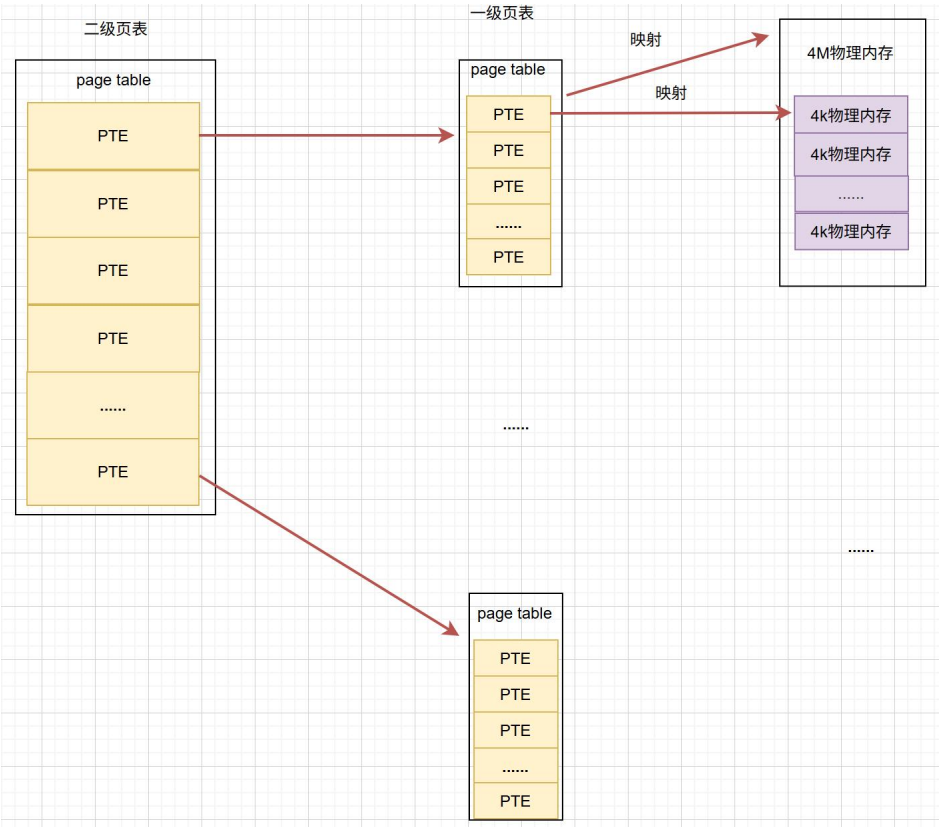
4、进程对系统内存的访问具有明显的局部性，在任意时刻，只需要为进程分配很少的内存就能保证进程的正常运行。

以 4G 的物理内存为例，根据局部性原理我们知道，进程在启动之后的任意时刻都不可能一下子就要访问全部的 4G 物理内存，但是需要给进程提供寻址 4G 物理内存的能力。所以在单级页表的情况下，必须要为进程额外分配 4M

的连续物理内存来存放 1024 张页表，不管进程访问不访问，这 4M 的开销是不能省的。

12.4.1 二级页表体系结构

再拿出一个 4K 的物理内存页作为页表，然后将这个页表放在单级页表的前面，组成一个二级页表的体系，如下图所示。



页表的本质是一个物理内存页，页表是 PTE 的数组，而 PTE 的本质是指向其映射的一个物理内存页，既然 PTE 可以指向一个普通的物理内存页，那么它也可以指向一个页表。所以，二级页表中的一个 PTE 本质上指向的还是一个物理内存页，只不过这个物理内存页比较特殊，它是一张页表（一级页表），一级页表是用来映射真正的物理内存的，一张一级页表可以映射 4M 物理内存。

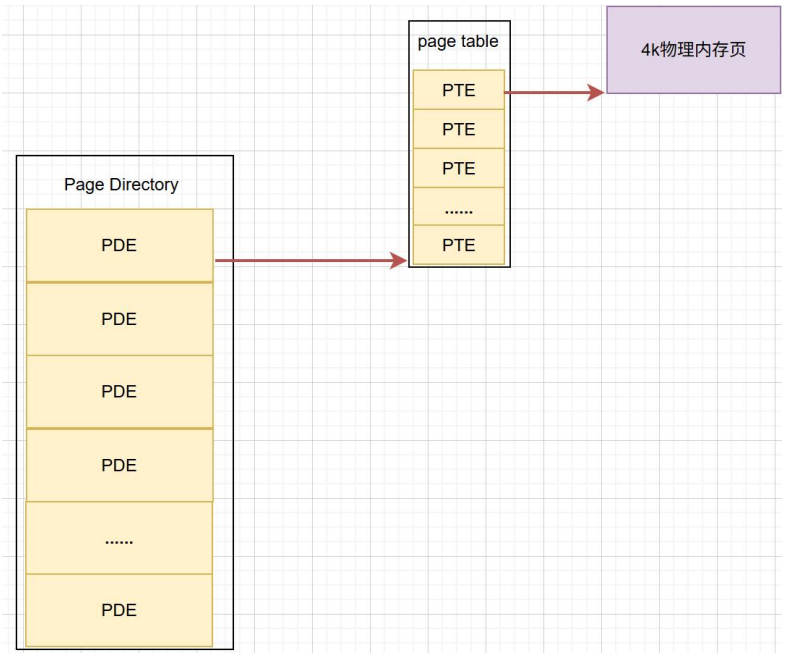
也就是说二级页表中的一个 PTE 就可以映射 4M 物理内存，同样的道理，二级页表中也包含了 1024 个 PTE，所以一张二级页表就可以映射 4G 的物理内存。

虽说二级页表和一级页表本质上都是一样的，它们都是一个物理内存页，但是习惯上将二级页表叫做页目录表，用来做一级页表的索引，二级页表中的 PTE 习惯上称为做页目录项 (Page Directory Entry, PDE)。

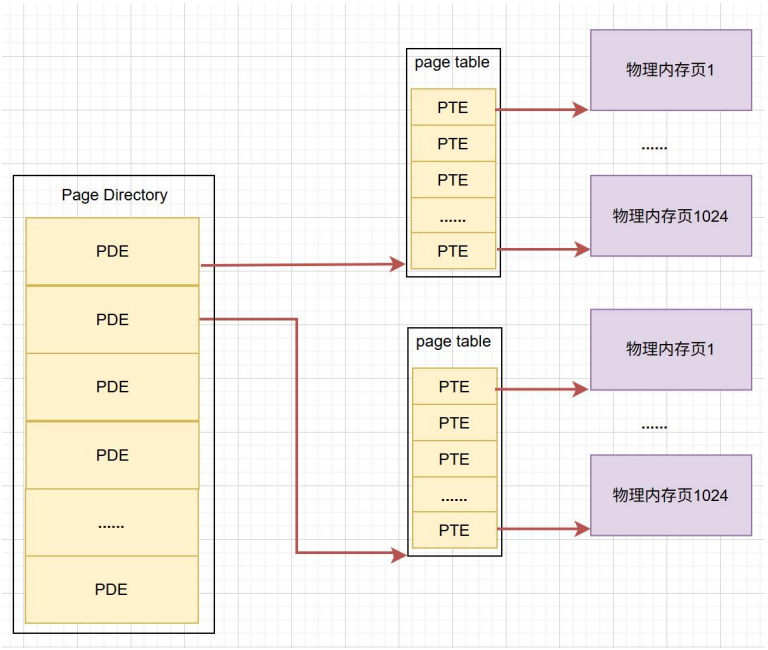
因为一张页目录表就可以映射 4G 的物理内存了，所以在二级页表的情况下，我们只需要在进程启动的时候额外为它分配 4K 的连续物理内存就可以了，这相比单级页表下，需要为每个进程额外分配 4M 的连续物理内存节省了非常多的内存资源。

但进程运行起来肯定会访问内存，要访问内存就需要有映射，在运行过程中光有一张页目录表肯定是不够的，根据程序局部性原理，进程在运行中的任意时刻，只会访问很小一块的内存，比如这时进程需要访问 4K 的物理内存（一个物理内存页），在二级页表情况下，内核会本着访问多少，映射多少的原则来进行内存映射。

当前系统中，进程只有一张页目录表，页目录表里的 PDE 没有映射任何东西，这时进程需要访问一个物理内存页，而对物理内存页的映射任务主要是在一级页表的 PTE 中，所以现在首要的任务就是建立一张一级页表出来，并用页目录表索引起来。



在二级页表的情况下，内核只需要一张 4K 的页目录表和一张 4K 的一级页表总共 8K 的内存就可以支持进程访问一个 4K 物理页面了，而根据程序的空间局部性原理，在不久的将来，进程只会访问与该物理内存页临近的页面，所以事实上，即使进程访问 4M 的内存，依然只需要一张 4K 的页目录表和一张 4K 的一级页表就可以满足了。这时候内核就需要一张页目录表和两张一级页表共 12K 额外的物理内存来映射，这依然比单级页表的 4M 连续物理内存开销小很多。

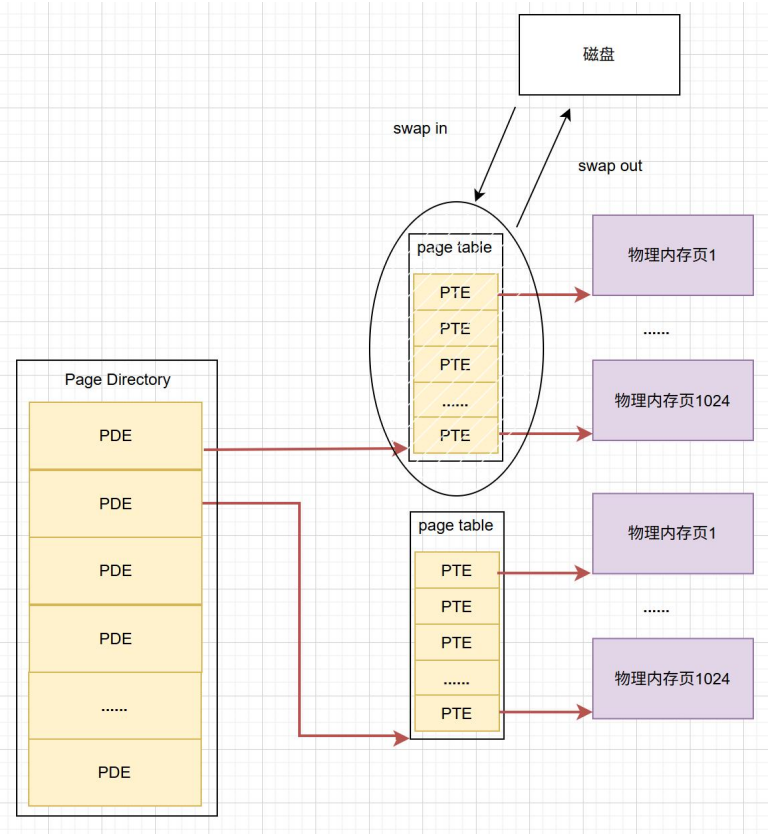


同理，随着进程一个 4M 接着一个 4M 物理内存的访问，在极端的情况下整个页目录表都被映射满了，这时候内核就需要 4K（页目录表）+ 4M（1024 张一级页表）的额外内存来保存映射关系了，这种情况下看起来会比单级页表下的 4M 内存开销大了那么一点点，但这种属于极端情况，非常少见，极大部分情况下还是比单级页表开销少很多。

而且在二级页表体系下，上面极端情况中的这 1024 张一级页表不需要是连续的，因为我们只需要保证顶级页表（这里指页目录表）是连续的就可以了，通过页目录表中的 PDE 可以唯一索引到一张一级页表的起始物理内存地址，而页表内的 4K 物理内存是连续的，所以依然可以通过数组的方式索引到一级页表中的 PTE，近而找到其映射的物理内存页面。

除此之外二级页表体系还有一个优势，就是当内存紧张的时候，那些不经常使用的一级页表可以被 swap out 到磁盘中，当进程再次访问到该页表映射的物理内存时，内核在将页表从磁盘中 swap in 到内存中。当然了，顶级页表（这里指页目录表）必须是常驻内存的，不允许 swap 。

既然页表的本质是一个物理内存页，那么同理，进程经常访问的那些页表也会被缓存到 CPU 高速缓存中加速下一次的访问速度。

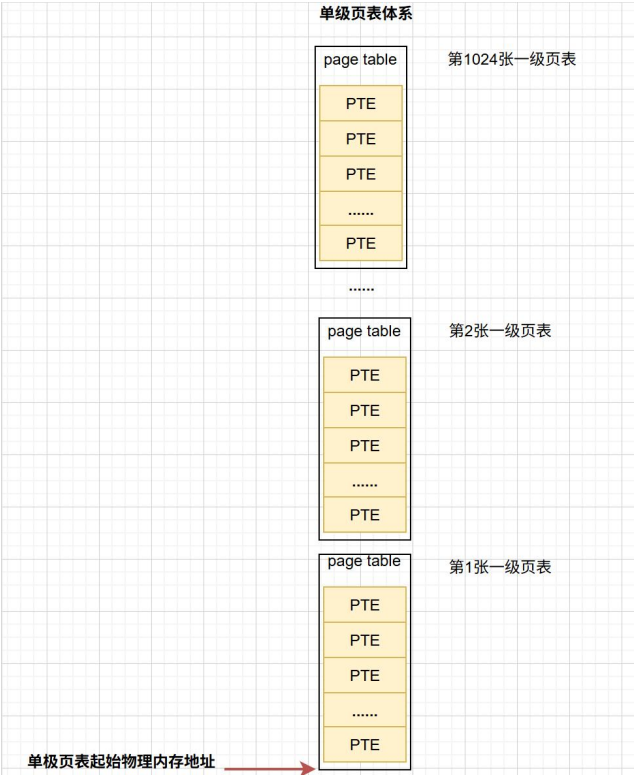


单级页表寻址的本质核心逻辑：

1、首先无论是几级页表，它们通过虚拟内存寻址的本质就是定位虚拟内存页对应在页表中的 PTE，然后通过 PTE 找到其映射的具体物理内存页。

2、页表的本质是一个物理内存页，其中包含了 1024 个 PTE，每个 PTE 可以映射一个具体的物理内存页，PTE 中保存了物理内存页的起始地址，进程地址空间中的一个虚拟内存页对应页表中的一个 PTE。因为在内核中是按照页为单位进行内存映射的。

3、在单级页表体系下，1024 张一级页表背后是通过连续的 4M 物理内存保存的，既然是连续的，那么我们可以把单级页表看做一个大的 PTE 数组，只要知道了单级页表的起始物理内存地址以及虚拟内存页对应 PTE 在单级页表（1024 张一级页表）中的 index，就可以定位到 PTE 。

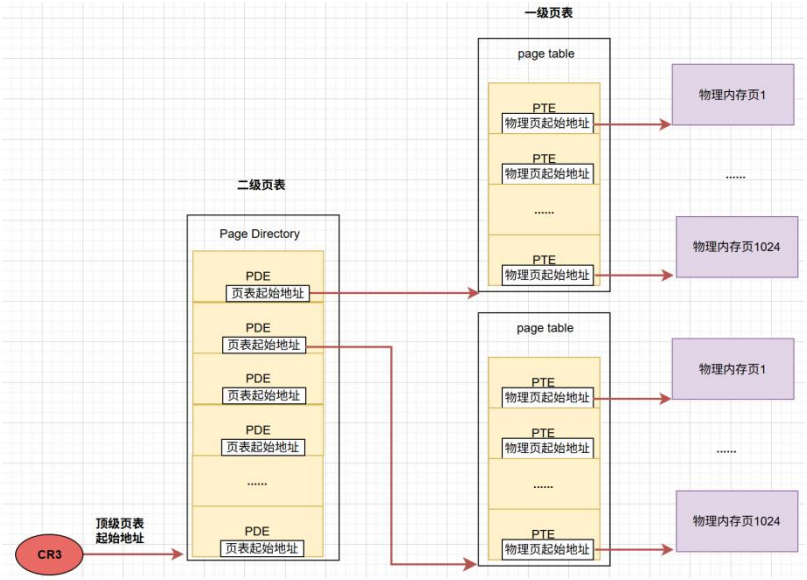


4、cr3 寄存器保存了顶级页表的起始物理内存地址，顶级页表随着进程的创建而创建，保存在进程 mm_struct->pgd，当进程被 CPU 调度的时候，会伴随着进程上下文切换，将 mm_struct->pgd 转换为物理内存地址并加载到 cr3 寄存器中。单级页表体系结构下，顶级页表指的就是单级页表（1024 张一级页表）。

5、无论在几级页表体系下，进程虚拟内存空间中的虚拟内存地址格式在设计上总共分为两大部分，一部分是用来定位虚拟内存页在页表中对应的 PTE 偏移，另一部分是用来定位物理内存页中要访问的具体字节。

6、有了单级页表的起始物理内存地址（保存在 cr3 寄存器中），有了虚拟内存页在单级页表中对应 PTE 的偏移(保存在虚拟内存地址中)，通过公式 页表起始地址 + 页表内偏移 * sizeof(PTE) 就可以定位到虚拟内存页对应的 PTE 了，而 PTE 中保存了映射物理内存页的起始地址，在加上虚拟内存地址中保存的物理内存页内偏移，这样就可以定位到虚拟内存地址对应的物理字节了。

从单级页表演进到二级页表之后，虚拟内存寻址的底层逻辑还是一样的，只不过顶级页表变成了页目录表（Page Directory），cr3 寄存器现在存放的是页目录表的起始物理内存地址。



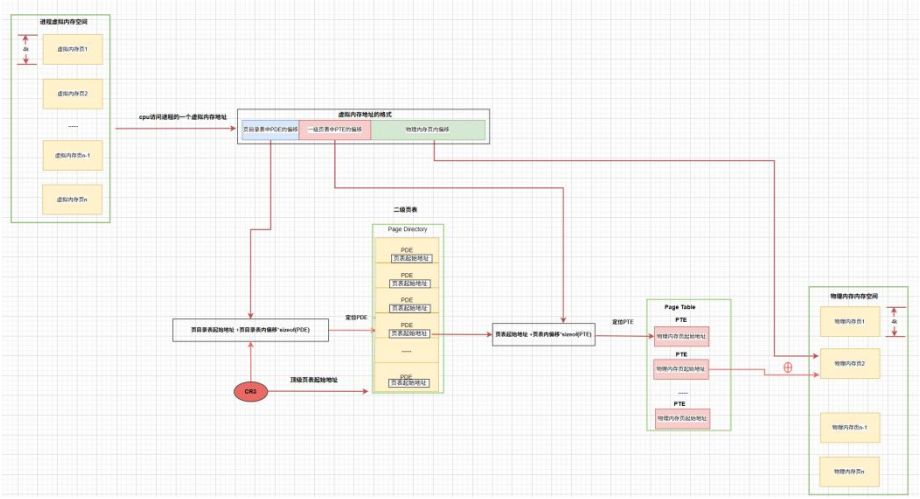
通过虚拟内存地址定位 PTE 的步骤由原来的一步变成了两步，因为多了一级页目录表，所以现在需要首先定位页目录表中的 PDE，然后通过 PDE 定位到具体的页表，近而找到页表中的 PTE。

所以在二级页表体系下的虚拟内存地址的格式也就发生了变化，单级页表下虚拟内存地址中只需要保存页表中的 PTE 偏移即可，二级页表下虚拟内存地址还需要多保存一份页目录表中 PDE 的偏移。

以 32 位操作系统为例。页目录表中的 PDE 和页表中的 PTE 均占用 4 字节。页目录表和页表的本质其实就是一个物理内存页，它们分别占用 4K 大小。因此一张页目录表中有 1024 个 PDE，要寻址这 1024 个 PDE 用 10 个 bit 就可以了，所以在上图中的虚拟内存地址中的 页目录表中 PDE 偏移 部分占用 10 个 bit 位。同样的道理，一张页表中有 1024 个 PTE, 要寻址这个 1024 个 PTE 也是需要 10 个 bit，上图中虚拟内存地址中的 一级页表中 PTE 偏移 部分也需要占用 10 个 bit 位。

可以通过虚拟内存地址中的前 10 个 bit 定位到页目录表中的 PDE ，而 PDE 指向的是一级页表的起始物理内存地址，然后通过接下来的 10 个 bit 就可以定位到页表中的 PTE，而 PTE 指向的是虚拟内存页最终映射的物理内存页的起始地址。然后，再使用上图中虚拟内存地址中的最后一部分 物理内存页内偏移找到物理内存页中要访问的那个字节，因为一个物理内存页占用 4K 大小，用 12 位 bit 就可以寻址内存页中的任意字节了。刚好可以组成一个 32 位的虚拟内存地址。

32 位操作系统为例，二级页表体系下的寻址过程如下：



1、当 CPU 访问进程虚拟内存空间中的一个地址时，会先从 cr3 寄存器中拿出页目录表的起始物理内存地址，然后从虚拟内存地址中解析出前 10 bit 的内容作为页目录表中 PDE 的偏移，通过公式 页目录表起始地址 + 页目录表内偏移 * sizeof(PDE) 就可以定位到该虚拟内存页在页目录表中的 PDE 了。

2、PDE 中保存了其指向的一级页表的起始物理内存地址，我们在从虚拟内存地址中解析出下一个 10 bit 作为页表中 PTE 的偏移，然后通过公式 页表起始地址 + 页表内偏移 * sizeof(PTE) 就能定位到虚拟内存页在一级页表中的 PTE 了。

12.4.2 32 位操作系统页表项 PTE 的结构

在进程的虚拟内存空间中，每一个虚拟内存页在页表中都有一个 PTE 与之对应，在 32 位系统中，每个 PTE 占用 4 个字节大小，其中保存了虚拟内存页背后映射的物理内存页的起始地址，以及进程访问物理内存的一些权限标识位。

```
typedef unsigned long   pteval_t;
typedef struct { pteval_t pte; } pte_t;
```

32 bit (4 字节) 下 PTE 布局格式：

bit位	31 : 12	11 : 9	8	7	6	5	4	3	2	1	0
	物理页起始内存地址	ignored	G	PAT	D	A	PCD	PWT	U/S	R/W	P

内核将整个物理内存划分为一页一页的单位，每个物理内存页大小为 4K，所以物理内存页的起始地址都是按照 4K 对齐的，所以物理内存页的起始地址的后 12 位全部是 0，我们只需要在 PTE 中存储物理内存地址的高 20 位就可以了，剩下的低 12 位可以用来标记一些权限位。下面是 PTE 权限位的含义：

P(0) 表示该 PTE 映射的物理内存页是否在内存中，值为 1 表示物理内存页在内存中驻留，值为 0 表示物理内存页不在内存中，可能被 swap 到磁盘上了。当 PTE 中的 P 位为 0 时，上图中的其他权限位将变得没有意义，这种情

况下其他 bit 位存放物理内存页在磁盘中的地址。当物理内存页需要被 swap in 的时候，内核会在这里找到物理内存页在磁盘中的位置。

R/W(1) 表示进程对该物理内存页拥有的读，写权限，值为 1 表示进程对该物理页拥有读写权限，值为 0 表示进程对该物理页拥有只读权限，进程对只读页面进行写操作将触发 page fault （写保护中断异常），用于写时复制（Copy On Write， COW）的场景。

比如，父进程通过 fork 系统调用创建子进程之后，父子进程的虚拟内存空间完全是一模一样的，包括父子进程的页表内容都是一样的，父子进程页表中的 PTE 均指向同一物理内存页面，此时内核会将父子进程页表中的 PTE 均改为只读的，并将父子进程共同映射的这个物理页面引用计数 + 1。

当父进程或者子进程对该页面发生写操作的时候，假设子进程先对页面发生写操作，随后子进程发现自己页表中的 PTE 是只读的，于是产生写保护中断，子进程进入内核态，在内核的缺页中断处理程序中发现，访问的这个物理页面引用计数大于 1，说明此时该物理内存页面存在多进程共享的情况，于是发生写时复制（Copy On Write， COW），内核为子进程重新分配一个新的物理页面，然后将原来物理页中的内容拷贝到新的页面中，最后子进程页表中的 PTE 指向新的物理页面并将 PTE 的 R/W 位设置为 1，原来物理页面的引用计数 - 1。

后面父进程在对页面进行写操作的时候，同样也会发现父进程的页表中 PTE 是只读的，也会产生写保护中断，但是在内核的缺页中断处理程序中，发现访问的这个物理页面引用计数为 1 了，那么就只需要将父进程页表中的 PTE 的 R/W 位设置为 1 就可以了。

U/S(2) 值为 0 表示该物理内存页面只有内核才可以访问，值为 1 表示用户空间的进程也可以访问。

PCD(4) 是 Page Cache Disabled 的缩写，表示 PTE 指向的这个物理内存页中的内容是否可以被缓存在 CPU CACHE 中，值为 1 表示 Disabled，值为 0 表示 Enabled。

PWT(3) 也是和 CPU CACHE 相关的控制位，Page Write Through 的缩写，值为 1 表示 CPU CACHE 中的数据发生修改之后，采用 Write Through 的方式同步回物理内存页中。值为 0 表示采用 Write Back 的方式同步回物理内存页。

当 CPU 修改了高速缓存中的数据之后，这些修改后的缓存内容同步回内存的方式有两种：

1、Write Back：CPU 修改后的缓存数据不会立马同步回内存，只有当 cache line 被替换时，这些修改后的缓存数据才会被同步回内存中，并覆盖掉对应物理内存页中旧的数据。

2、Write Through：CPU 修改高速缓存中的数据之后，会立刻被同步回物理内存页中。

A(5) 表示 PTE 指向的这个物理内存页最近是否被访问过，1 表示最近被访问过（读或者写访问都会设置为 1），0 表示没有。该 bit 位被硬件 MMU 设置，由操作系统重置。内核会经常检查该比特位，以确定该物理内存页的活跃程度，不经常使用的内存页，很可能就会被内核 swap out 出去。

D(6) 主要针对文件页使用，当 PTE 指向的物理内存页是一个文件页时，进程对这个文件页写入了新的数据，这时文件页就变成了脏页，对应的 PTE 中 D 比特位会被设置为 1，表示文件页中的内容与其背后对应磁盘中的文件内容不同步了。

G(8) 设置为 1 表示该 PTE 是全局的，该标志位表示 PTE 中保存的映射关系是否是全局的。一般来说进程都有各自独立的虚拟内存空间，进程的页表也是独立的 ，CPU 每次访问进程虚拟内存地址的时候都需要进行地址翻译，为了加速地址翻译的速度，避免每次遍历页表，CPU 会把经常被访问到的 PTE 缓存在一个 TLB 的硬件缓存中，由于 TLB 中缓存的是当前进程相关的 PTE，所以操作系统每次在切换进程的时候，都会重新刷新 TLB 缓存。

有一些 PTE 是所有进程共享的，比如说内核虚拟内存空间中的映射关系，所有进程进入内核态看到的都是一样的。所以会将这些全局共享的 PTE 中的 G 比特位置为 1 ，这样在每次进程切换的时候，就不会 flush 掉 TLB 缓存的那些共享的全局 PTE（比如内核地址的空间中使用的 PTE）,从而在很大程度上提升了性能。

PTE 相关的权限比特位定义在内核文件

arch\x86\include\asm\pgtable_types.h 中：

```
#define _PAGE_BIT_PRESENT   0   /* is present */
#define _PAGE_BIT_RW        1   /* writeable */
#define _PAGE_BIT_USER      2   /* userspace addressable */
#define _PAGE_BIT_PWT       3   /* page write through */
#define _PAGE_BIT_PCD       4   /* page cache disabled */
#define _PAGE_BIT_ACCESSED  5   /* was accessed (raised by CPU) */
#define _PAGE_BIT_DIRTY     6   /* was written to (raised by CPU) */
#define _PAGE_BIT_PSE       7   /* 4 MB (or 2MB) page */
#define _PAGE_BIT_PAT       7   /* on 4KB pages */
#define _PAGE_BIT_GLOBAL    8   /* Global TLB entry PPro+ */

// 从 PTE 中提取相应比特位的掩码
#define _PAGE_PRESENT   (_AT(pteval_t, 1) << _PAGE_BIT_PRESENT)
#define _PAGE_RW    (_AT(pteval_t, 1) << _PAGE_BIT_RW)
#define _PAGE_USER  (_AT(pteval_t, 1) << _PAGE_BIT_USER)
#define _PAGE_PWT   (_AT(pteval_t, 1) << _PAGE_BIT_PWT)
#define _PAGE_PCD   (_AT(pteval_t, 1) << _PAGE_BIT_PCD)
#define _PAGE_ACCESSED  (_AT(pteval_t, 1) << _PAGE_BIT_ACCESSED)
#define _PAGE_DIRTY (_AT(pteval_t, 1) << _PAGE_BIT_DIRTY)
#define _PAGE_PSE   (_AT(pteval_t, 1) << _PAGE_BIT_PSE)
#define _PAGE_GLOBAL    (_AT(pteval_t, 1) << _PAGE_BIT_GLOBAL)
```

除此之外，内核还定义了一系列的方法，用于操作 PTE 的相关权限比特位，这些方法三个一组，分别用于相应权限比特位的查询，设置，清除操作，定义在内核文件 arch\x86\include\asm\pgtable.h

```
// 查询 PTE 指向的物理内存页是否在内存中
static inline int pte_present(pte_t a)
{
    return pte_flags(a) & (_PAGE_PRESENT | _PAGE_PROTNONE);
}
// 查询内存页是否可写
```

```c
static inline int pte_write(pte_t pte)
{
    /*
     * Shadow stack pages are logically writable, but do not have
     * _PAGE_RW.  Check for them separately from _PAGE_RW itself.
     */
    return (pte_flags(pte) & _PAGE_RW) || pte_shstk(pte);
}
// 设置内存页可写
static inline pte_t pte_mkwrite(pte_t pte, struct vm_area_struct *vma)
{
    return pte_mkwrite_novma(pte);
}

// 禁止读写
static inline pte_t pte_wrprotect(pte_t pte)
{
    pte = pte_clear_flags(pte, _PAGE_RW);
    return pte_mksaveddirty(pte);
}

// 查询 PTE 是否是 global
static inline int pte_global(pte_t pte)
{
    return pte_flags(pte) & _PAGE_GLOBAL;
}
static inline pte_t pte_mkglobal(pte_t pte)
{
    return pte_set_flags(pte, _PAGE_GLOBAL);
}
static inline pte_t pte_mkdirty(pte_t pte)
{
    pte = pte_set_flags(pte, _PAGE_DIRTY | _PAGE_SOFT_DIRTY);

    return pte_mksaveddirty(pte);
}

// 查询 PTE 指向的内存页是否是脏页
static inline bool pte_dirty(pte_t pte)
{
    return pte_flags(pte) & _PAGE_DIRTY_BITS;
}

static inline pte_t pte_mkclean(pte_t pte)
```

```
{
    return pte_clear_flags(pte, _PAGE_DIRTY_BITS);
}

static inline pte_t pte_mkdirty(pte_t pte)
{
    pte = pte_set_flags(pte, _PAGE_DIRTY | _PAGE_SOFT_DIRTY);

    return pte_mksaveddirty(pte);
}

// 查询 PTE 指向的内存页是否最近被访问过
static inline int pte_young(pte_t pte)
{
    return pte_flags(pte) & _PAGE_ACCESSED;
}
static inline pte_t pte_mkold(pte_t pte)
{
    return pte_clear_flags(pte, _PAGE_ACCESSED);
}
static inline pte_t pte_mkyoung(pte_t pte)
{
    return pte_set_flags(pte, _PAGE_ACCESSED);
}
```

12.4.3 32 位操作系统页表项 PDE 的结构

PDE 用来指向一级页表的起始物理内存地址，而页表的本质是一个物理内存页（4K 大小），因此页表的起始内存地址也是按照 4K 对齐的，后 12 位全部为 0 ，可以用 PDE 的低 12 位来标记页目录项的权限位：

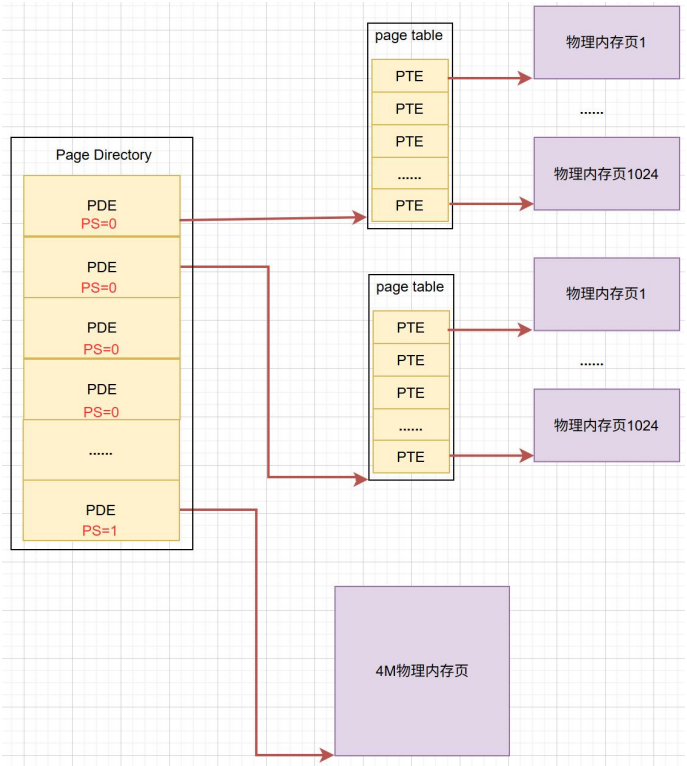
```
typedef unsigned long   pgdval_t;
typedef struct { pgdval_t pgd; } pgd_t;
```

和页表中 PTE 的权限位不同的是，PDE 中没有了第 6 个比特位脏页标记位，因为 PDE 指向的是一级页表，页表并不是一个文件页，所以脏页标记在这里就没有意义了。PDE 中也没了第 8 比特位，Global 全局标记位，因为 TLB 缓存的是 PTE 而不是 PDE，所以不需要设置 Global 标记来防止进程切换导致 TLB flush。不同的是 PDE 中的第 7 比特位由 PTE 中的 PAT 标记变为了 PS 标记位。

当 PS 标记为 0 的时候，PDE 指向一级页表的起始内存地址，这种情况下，PDE 就起到二级页表的作用。当 PS 标记为 1 的时候，PDE 就会被内核拿来当

做 PTE 使用，不过这个 ”PTE“ 比较特殊，其特殊之处在于该 PDE 会指向一个大页内存，这个物理内存页不是普通的 4K 大小，而是 4M 大小。

bit位	31 : 12	11 : 8	7	6	5	4	3	2	1	0
	物理页起始内存地址	ignored	PS	ignored	A	PCD	PWT	U/S	R/W	P



既然这样，PDE 也可以直接指向一张 4M 的内存大页。为什么内核还需要支持大页内存呢？

Linux 管理内存的最小单位是 page，每个 page 描述 4K 大小的物理内存，但在一些内存敏感的使用场景中，用户往往期望使用一些巨型大页。因为这些巨型页要比普通的 4K 内存页要大很多，所以遇到缺页中断的情况就会相对减少，由于减少了缺页中断所以性能会更高。

另外，由于巨型页比普通页要大，所以巨型页需要的页表项要比普通页要少，页表项里保存了虚拟内存地址与物理内存地址的映射关系，当 CPU 访问内存的时候需要频繁通过 MMU 访问页表项获取物理内存地址，由于要频繁访问，所以页表项一般会缓存在 TLB 中，因为巨型页需要的页表项较少，所以节约了 TLB 的空间同时降低了 TLB 缓存 MISS 的概率，从而加速了内存访问。

还有一个使用巨型页受益场景，当一个内存占用很大的进程，通过 fork 系统调用创建子进程的时候，会拷贝父进程的相关资源，其中就包括父进程的页表，由于巨型页使用的页表项少，所以拷贝的时候性能会提升很多。

PS 标记为 1 的情况下，PDE 指向的是一个 4M 的物理大页内存，这种情况下内核就把 PDE 当做一个特殊的 ”PTE“ 使用，所以 PDE 中的比特位布局又发生了变化，不过大部分还是和 PTE 一样的。

有一点需要注意，第 13 到 31 比特位的作用，粗略的从总体来讲这个范围的比特位确实是用来保存 4M 大页的起始内存地址，但其实，M 内存大页的起始地址都是按照 4M 对齐的，也就是说 4M 大页的起始内存地址的后 22 位全部为 0 ，只需要用 10 个比特位就可以标记了，事实上，4M 大页的起始内存地址在内核中就是使用 31:22 范围内的比特标记的，剩下的比特用来做内存地址的扩展使用。

和 PTE 一样，PDE 的相关权限比特位也定义在内核文件：arch\x86\include\asm\pgtable_types.h 中：

```
#define _PAGE_BIT_PSE        7
// 从 PDE 中提取 PS 比特位掩码
#define _PAGE_PSE    (_AT(pteval_t, 1) << _PAGE_BIT_PSE)
```

并通过如下位运算，从 PDE 中提取 PS 比特位来确定该 PDE 指向的是一级页表还是 4M 大页。

12.4.4 四级页表结构

在 32 位系统中，内核主要采用二级页表体系来进行虚拟内存寻址，但是到了 64 位系统中，二级页表明显就不够用了，因为二级页表最多只能映射 4G 的物理内存空间，而 64 位系统中，进程的虚拟寻址空间是巨大的，进程的用户态需要寻址 128T 的虚拟内存空间，内核态也有 128T 的虚拟内存空间。

为了寻址这么大的虚拟内存空间，内核在 64 位系统中引入了四级页表体系，64 位系统中的四级页表相比 32 位系统中的二级页表来说，多引入了两个层级的页目录，分别是四级页表和三级页表。

内核中第四级页表为全局页目录 PGD（Page Global Directory），PGD 中的页目录项叫做 pgd_t，PGD 是四级页表体系下的顶级页表，保存在进程 struct mm_struct 结构中的 pgd 属性中，在进程调度上下文切换的时候，由内核通过 load_new_mm_cr3 方法将 pgd 中保存的顶级页表虚拟内存地址转换物理内存地址，随后加载到 cr3 寄存器中，从而完成进程虚拟内存空间的切换。

第三级页表在内核中称之为上层页目录 PUD（Page Upper Directory），PUD 中的页目录项叫做 pud_t 。

第二级页表叫做中间页目录 PMD（Page Middle Directory），PMD 中的页目录项叫做 pmd_t，最底层的用来直接映射物理内存页面的一级页表，名字不变还叫做页表（Page Table）。

由于在四级页表体系下，又多引入了两层页目录（PGD,PUD）,所以导致其通过虚拟内存地址定位 PTE 的步骤又增加了两步，首先需要定位顶级页表 PGD 中的页目录项 pgd_t，pgd_t 指向的 PUD 的起始内存地址，然后在定位 PUD 中的页目录项 pud_t，后面的流程就和二级页表一样。

因此 64 位的虚拟内存地址格式如下：

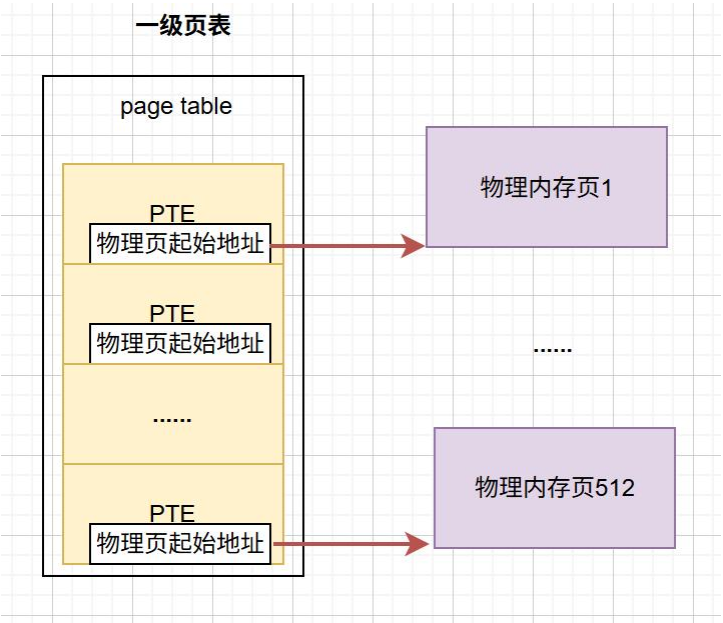
32 位系统中的页目录表，页表和 64 位系统中的页目录表，页表在内核中都是使用一个普通 4K 的物理内存页存储映射关系的，不同的是 64 位系统中的页表中的 PTE 以及页目录表（PGD,PUD,PMD）中的 PDE 都是占用 8 个字节，在内核中都是使用 unsigned long 类型描述：

```
//arch\x86\include\asm\pgtable_64_types.h
typedef unsigned long   pteval_t;
typedef unsigned long   pmdval_t;
typedef unsigned long   pudval_t;
typedef unsigned long   pgdval_t;

typedef struct { pteval_t pte; } pte_t;
typedef struct { pmdval_t pmd; } pmd_t;

arch\x86\include\asm\pgtable_types.h
typedef struct { pudval_t pud; } pud_t;
typedef struct { pgdval_t pgd; } pgd_t;
```

内核使用 struct 结构来包裹 unsigned long 类型的目的是要确保这些页目录项以及页表项只能被专门的辅助函数访问，不能直接访问。

```
/*
 * entries per page directory level
 */
#define PTRS_PER_PTE    512

/* PAGE_SHIFT determines the page size */
#define PAGE_SHIFT      12
```

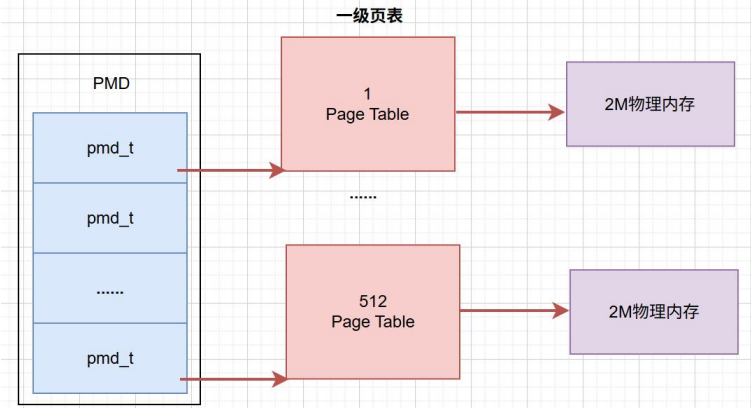
寻址页表中 512 个 PTE，需要用 9 个 bit ，因此虚拟内存地址中的 一级页表中的 PTE 偏移 占用 9 个 bit 位。而一个 PTE 可以映射 4K 大小的物理内存（一个物理内存页），所以在 64 位的四级页表体系下，一张一级页表可以映射的物理内存空间大小为 2M 大小。

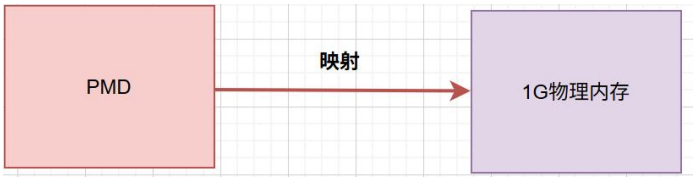


一张中间页目录 PMD 也是 4K 大小，PMD 中的页目录项 pmd_t 也是占用 8 个字节，所以一张 PMD 中只能容纳 512 个 pmd_t，内核中使用 PTRS_PER_PMD 常量来表示 PMD 中可以容纳多少个页目录项 pmd_t。因次 64 位虚拟内存地址中的 PMD 中的页目录偏移使用 9 个 bit 表示。

```
/*
 * PMD_SHIFT determines the size of the area a middle-level
 * page table can map
 */
#define PMD_SHIFT    21
#define PTRS_PER_PMD     512
```

一个 pmd_t 指向一张一级页表，所以一个 pmd_t 可以映射的物理内存为 2M，内核中使用 PMD_SHIFT 常量来表示一个 pmd_t 可以映射的物理内存范围：2^PMD_SHIFT。一张 PMD 可以映射 1G 的物理内存。

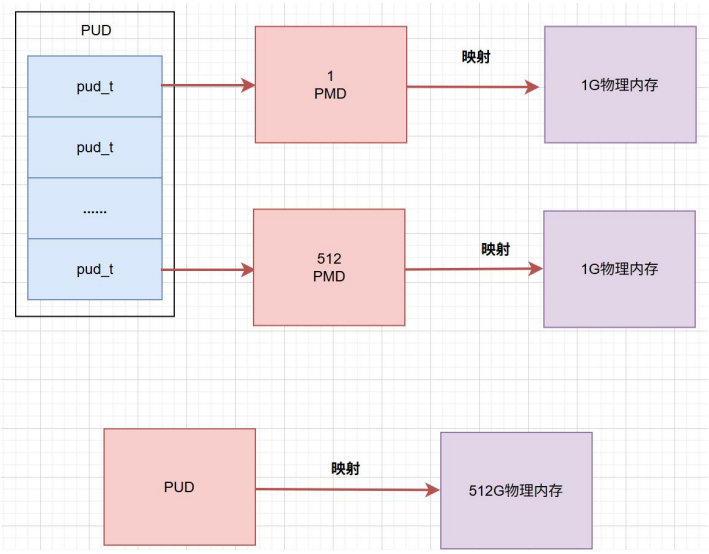




同理，一张上层页目录 PUD 中可以容纳 512 个页目录项 pud_t，内核中使用 PTRS_PER_PUD 常量来表示 PUD 中可以容纳多少个页目录项 pud_t。64 位虚拟内存地址中的 PUD 中的页目录偏移 也是使用 9 个 bit 表示。

```
/*
 * 3rd level page
 */
#define PUD_SHIFT   30
#define PTRS_PER_PUD    512
```

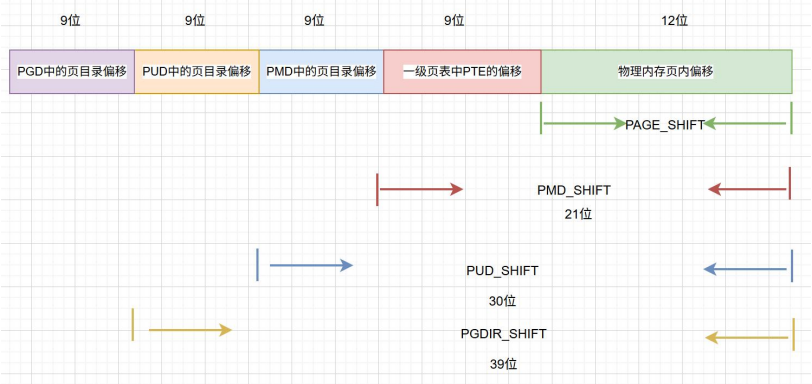
内核中使用 PUD_SHIFT 常量来表示一个 pud_t 可以映射的物理内存范围：2^PUD_SHIFT，一个 pud_t 指向一张 PMD，因此可以映射 1G 的物理内存。一张 PUD 可以映射 512G 的物理内存。



同理，顶级页目录 PGD 中可以容纳的页目录 pgd_t 个数 PTRS_PER_PGD = 512， 64 位虚拟内存地址中的 PGD 中的页目录偏移 也是使用 9 个 bit 就可以表示了，一个 pgd_t 可以映射的物理内存为 2^PGDIR_SHIFT = 512 G。一张 PGD 可以映射的物理内存为 256 T。

```
/*
 * PGDIR_SHIFT determines what a top-level page table entry can map
 */
#define PGDIR_SHIFT     39
#define PTRS_PER_PGD        512
```

因此，64 位虚拟内存地址中的比特位布局情况如下：



PAGE_SHIFT 用来表示页表中的一个 PTE 可以映射的物理内存大小（4K）。

PMD_SHIFT 用来表示 PMD 中的一个页目录项 pmd_t 可以映射的物理内存大小（2M）。

PUD_SHIFT 用来表示 PUD 中的一个页目录项 pud_t 可以映射的物理内存大小（1G）。

PGD_SHIFT 用来表示 PGD 中的一个页目录项 pgd_t 可以映射的物理内存大小（512G）。

这些 XXX_SHIFT 常量在内核中除了可以表示对应页目录项映射的物理内存大小之外，还可以从一个 64 位虚拟内存地址中获取其在对应页目录中的偏移。

例如，需要从一个 64 位虚拟内存地址中获取它在 PGD 页表中的偏移，可以将虚拟内存地址右移 PGD_SHIFT 位得到：

```
#define pgd_index(a)  (((a) >> PGDIR_SHIFT) & (PTRS_PER_PGD - 1))
```

然后，通过 PGD 的起始内存地址加上 pgd_index 得到虚拟内存地址在 PGD 页表中的页目录项 pgd_t。

```
static inline pgd_t *pgd_offset_pgd(pgd_t *pgd, unsigned long address)
{
    return (pgd + pgd_index(address));
};
```

同样的道理，就可以将虚拟内存地址右移 PUD_SHIFT 位，并用掩码 PTRS_PER_PUD - 1 掩掉高 9 位，只保留低 9 位，就可以得到虚拟内存地址在 PUD 中的偏移(PTRS_PER_PUD - 1 转换为二进制是 9 个 1，用来截取最低的 9 个比特位。)：

```
static inline unsigned long pud_index(unsigned long address)
{
    return (address >> PUD_SHIFT) & (PTRS_PER_PUD - 1);
}
```

通过 pgd_t 获取 PUD 的起始内存地址 + pud_index 得到虚拟内存地址对应的 pud_t：

```
static inline pud_t *pud_offset(p4d_t *p4d, unsigned long address)
{
    return p4d_pgtable(*p4d) + pud_index(address);
}
```

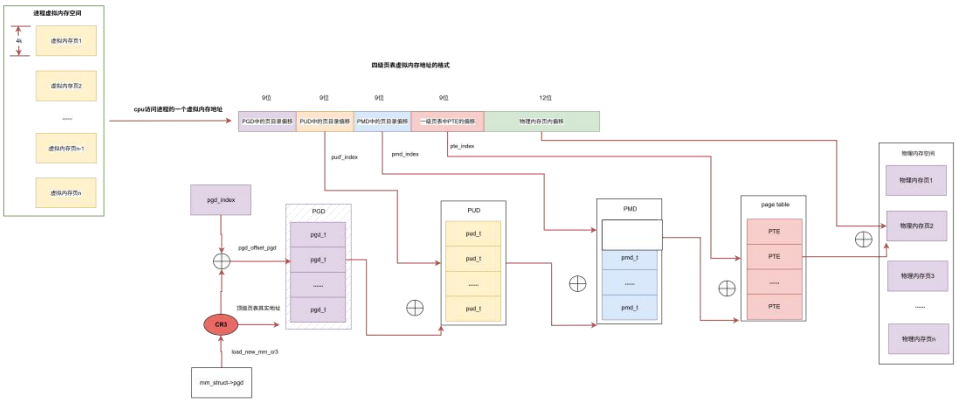
根据相同的计算逻辑，可以通过 pmd_offset 函数获取虚拟内存地址在 PMD 中的页目录项 pmd_t：

```
static inline pmd_t *pmd_offset(pud_t *pud, unsigned long address)
{
    return pud_pgtable(*pud) + pmd_index(address);
}

static inline unsigned long pmd_index(unsigned long address)
{
    return (address >> PMD_SHIFT) & (PTRS_PER_PMD - 1);
}
```

通过 pte_offset_kernel 函数可以获取虚拟内存地址在一级页表中的 PTE：

```
static inline pte_t *pte_offset_kernel(pmd_t *pmd, unsigned long address)
{
    return (pte_t *)pmd_page_vaddr(*pmd) + pte_index(address);
}

static inline unsigned long pte_index(unsigned long address)
{
    return (address >> PAGE_SHIFT) & (PTRS_PER_PTE - 1);
}
```

所以，四级页表体系的寻址过程如下：



1、首先 MMU 会从 cr3 寄存器中获取顶级页目录 PGD 的起始内存地址，然后通过 pgd_index 从虚拟内存地址中截取 PGD 中的页目录项偏移，这样就定位到了具体的一个 pgd_t。

2、pgd_t 中保存的是 PMD 的起始内存地址，通过 pud_index 可以从虚拟内存地址中截取 PUD 中的页目录项偏移，从而确定虚拟内存地址在 PUD 中的页目录项 pud_t。

3、同样的道理，根据 pud_t 中保存的 PMD 其实内存地址，在加上通过 pmd_index 获取到的 PMD 中的页目录项偏移，定位到虚拟内存地址在 PMD 中的页目录项 pmd_t。

4、后面的寻址流程就和二级页表一样，pmd_t 指向具体页表的起始内存地址，通过 pte_index 截取虚拟内存地址在 一级页表中的 PTE 偏移，最终定位到一个具体的 PTE 中，PTE 指向的正是虚拟内存地址映射的物理内存页面，然后通过虚拟内存地址中的低 12 位（物理内存页内偏移），最终确定到一个具体的物理字节上。

12.4.5 64 位系统下页表项 PTE 的结构

在 64 位系统中，页表中 PTE 在内核中使用 unsigned long 类型描述，占用 8 个字节：

```
typedef unsigned long   pteval_t;
typedef struct { pteval_t pte; } pte_t;
```

bit位	63	62 : 59	58 : 52	51 : 36	35 : 12	11	10 : 9	8	7	6	5	4	3	2	1	0
	XD	Protection	ignored	Reserved	4k物理内存页的物理地址	R	ignored	G	PAT	D	A	PCD	PWT	U/S	R/W	P

以 36 位物理内存地址（最多 52 位）为例进行说明，首先物理内存页的起始内存地址都是按照 4K 对齐的，所以 36 位物理内存地址的低 12 位全部为 0 ，和 32 位的 PTE 一样，内核用这低 12 位来描述 PTE 的权限位，其中 0 到 8 之间的比特位，在 32 位 PTE 和 64 位 PTE 中含义都是一样的

R(11) 这里的 R 表示 restart，该比特位主要用于 HLAT paging，当遍历到 R 位是 1 的 PTE 时，MMU 会重新从 CR3 寄存器开始遍历页表。

Reserved(51:36) 这些是预留位，全部设置为 0 。

Protection(62:59) 这 4 个比特位用于对物理内存页的访问进行控制。

XD(63) 该比特位是 64 位 PTE 中新增的，32 位 PTE 中是没有的，值为 1 表示该 PTE 所映射的物理内存页面中的数据是可以被执行的。

12.4.6 64 位系统下页目录项 PDE 的结构

在 64 位系统中使用的四级页表体系中一共包含了三个层级的页目录，它们分别为：全局页目录 PGD（Page Global Directory），上层页目录 PUD（Page Upper Directory），PMD（Page Middle Directory）。

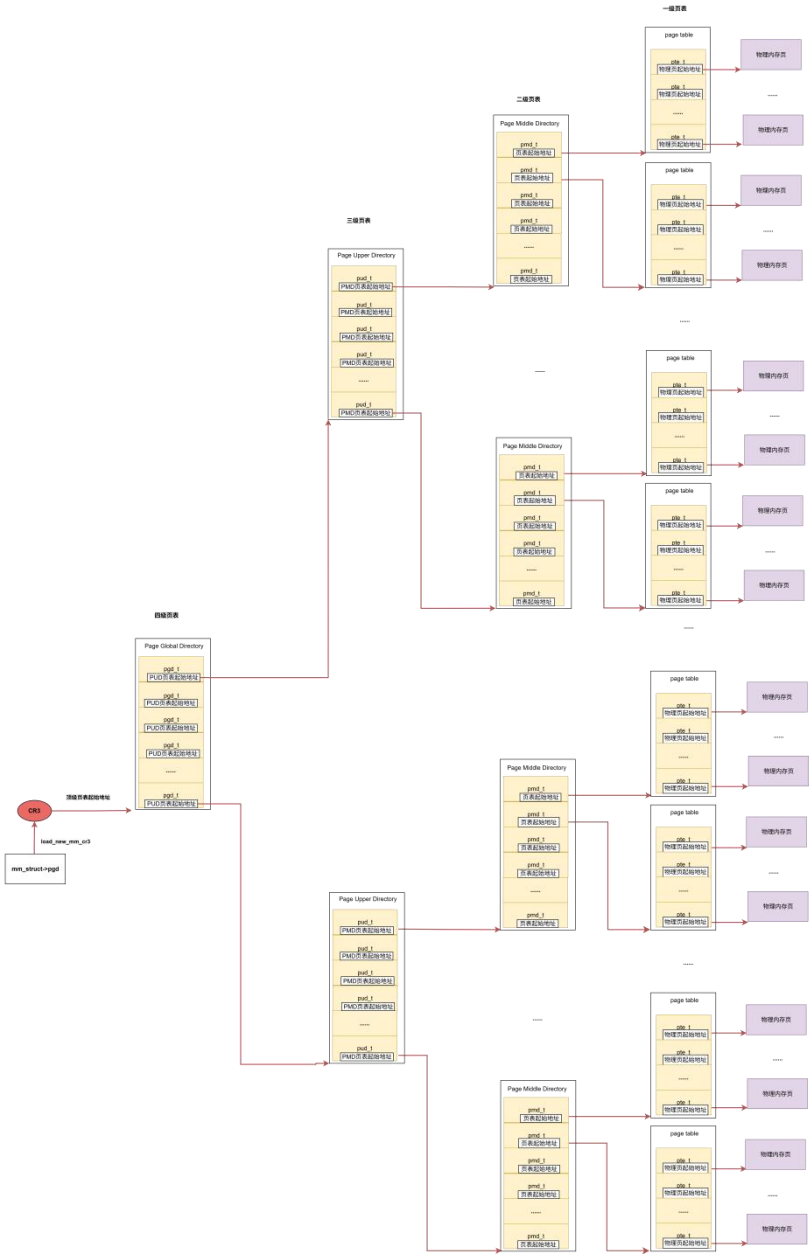
这三种类型的页目录中的页目录项 PDE 在内核中也是使用 unsigned long 类型来描述的，在 64 位系统中占用 8 个字节：

```
typedef unsigned long   pmdval_t;
typedef unsigned long   pudval_t;
typedef unsigned long   pgdval_t;

typedef struct { pmdval_t pmd; } pmd_t;
typedef struct { pudval_t pud; } pud_t;
typedef struct { pgdval_t pgd; } pgd_t;
```

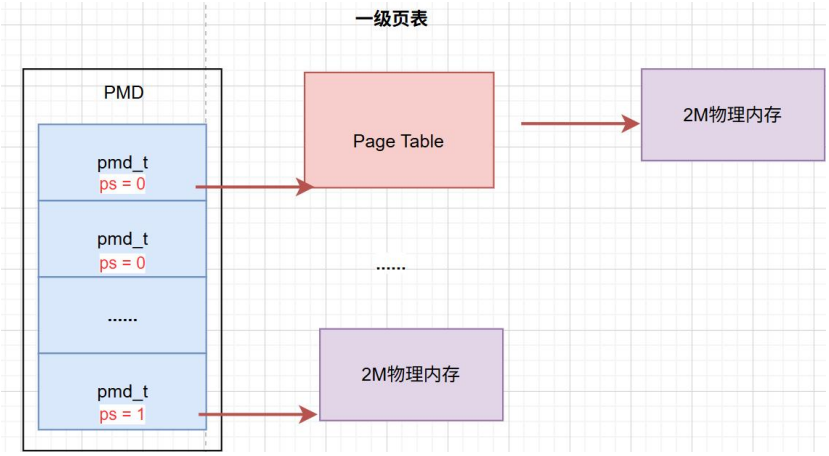
64 位 PDE 中的比特位布局如下图所示：

bit位	63	62 : 52	51 : 36	35 : 12	11	10 : 8	7	6	5	4	3	2	1	0
	XD	ignored	Reserved	下一级页目录或者页表的物理地址	R	ignored	PS	ignored	A	PCD	PWT	U/S	R/W	P

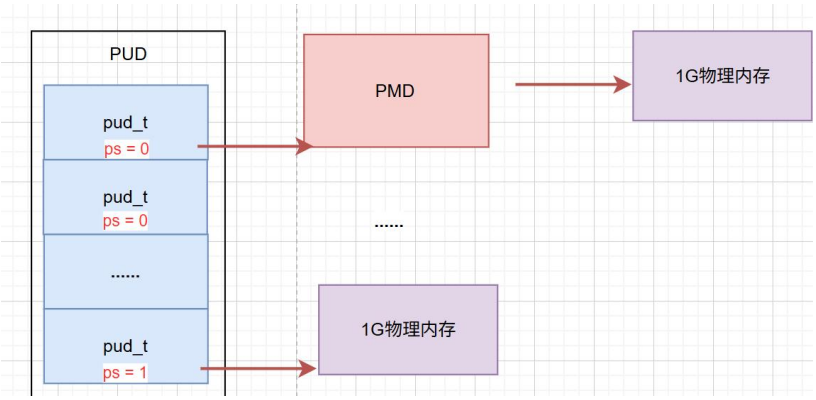
当 64 位 PDE 的 PS(7) 比特位为 0 时，该 PDE 指向的是其下一级页目录或者页表的起始内存地址。



当 64 位 PDE 的 PS(7) 比特位为 1 时，该 PDE 指向的就是一个内存大页，对于 PMD 中的页目录项 pmd_t 而言，它指向的是一张 2M 大小的物理内存大页。



对于 PUD 中的页目录项 pud_t 而言，它指向的是一张 1G 大小的物理内存大页。



当 64 位 PDE 的 PS(7) 比特位为 1 时，这些页目录项 PDE 被当做一个特殊的 ”PTE“ ，因此 PDE 中的比特位布局变成了 64 位 PTE 中的形式。

bit位	63	62 : 59	58 : 52	51 : 36	35 : 12	11	10 : 9	8	7	6	5	4	3	2	1	0
	XD	Protection	ignored	Reserved	大页内存的物理地址	R	ignored	G	1	D	A	PCD	PWT	U/S	R/W	P

另外，内核提供一系列的辅助函数来对页目录进行操作：

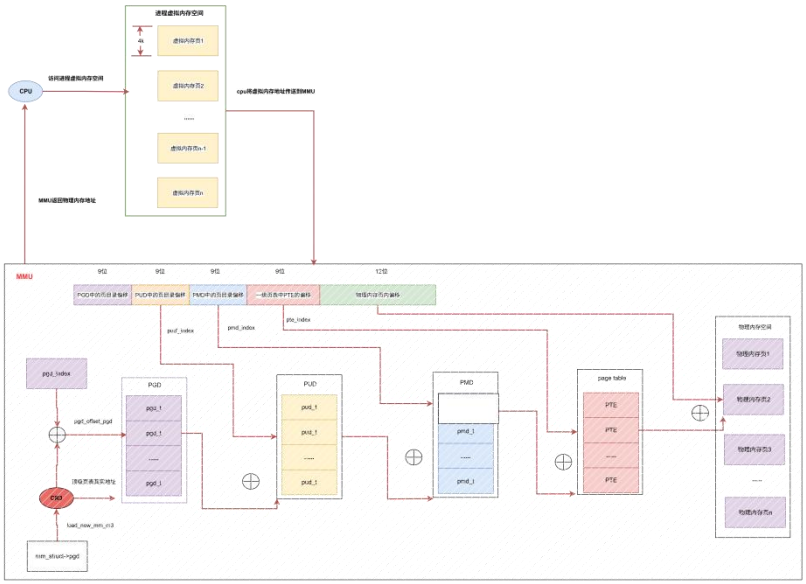
pgd_alloc，pud_alloc，pmd_alloc 负责创建初始化对应的页目录。

mk_pgd，mk_pud，mk_pmd，mk_pte 用于创建相应页目录项和页表项，并初始化上述比特位。

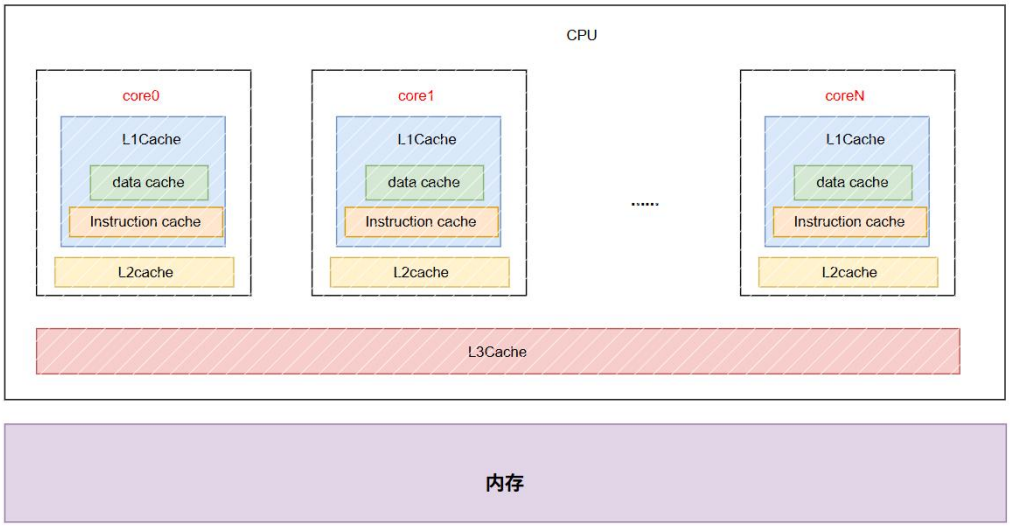
以及提供相关 pgd_xxx，pud_xxx，pmd_xxx 等形式的辅助函数，用于对相关比特位的增删改查操作。

12.5 CPU 的寻址过程

在 CPU 中引入了一个专门对页表进行遍历的地址翻译硬件 MMU（Memory Management Unit），MMU 硬件加速了整个地址翻译的过程。页表的整个遍历操作均是在 MMU 中进行的：



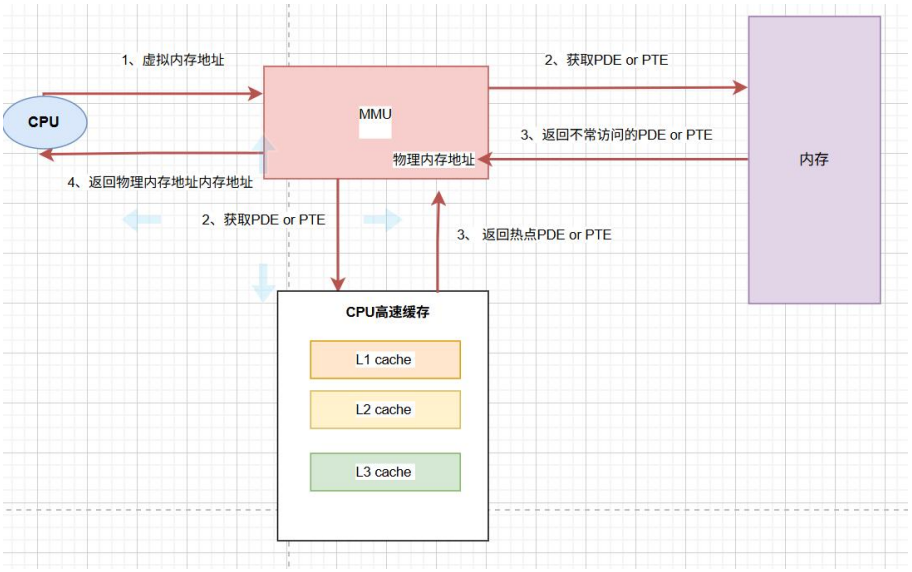
页目录，页表的本质其实在内核看来都是一张普通的 4K 大小的物理内存页，而物理内存页中经常被访问到的内存数据都是缓存在 CPU 的高速缓存 L1 ，L2，L3 CACHE 中的，所以可以利用局部性原理加速 CPU 对内存的访问。



所以页目录表和页表中那些经常被 MMU 遍历到的页目录项 PDE，页表项 PTE 均会缓存在 CPU 的 CACHE 中，这样 MMU 就可以直接从 CPU 高速缓存中获取 PDE , PTE 了，近一步加速了整个地址翻译的过程。

当 MMU 拿到一个 CPU 正在访问的虚拟内存地址之后， MMU 首先会从 CR3 寄存器中获取顶级页目录表 PGD 的起始内存地址，然后从虚拟内存地址中提取出 pgd_index，从而定位到 PGD 中的一个页目录项 pdg_t，MMU 首先会从 CPU 的高速缓存中去获取这个 pgd_t，如果 pgd_t 经常被访问到，那么此时它已经存在于高速缓存中，MMU 直接可以进行下一级页目录的地址翻译操作，避免了慢速的内存访问。

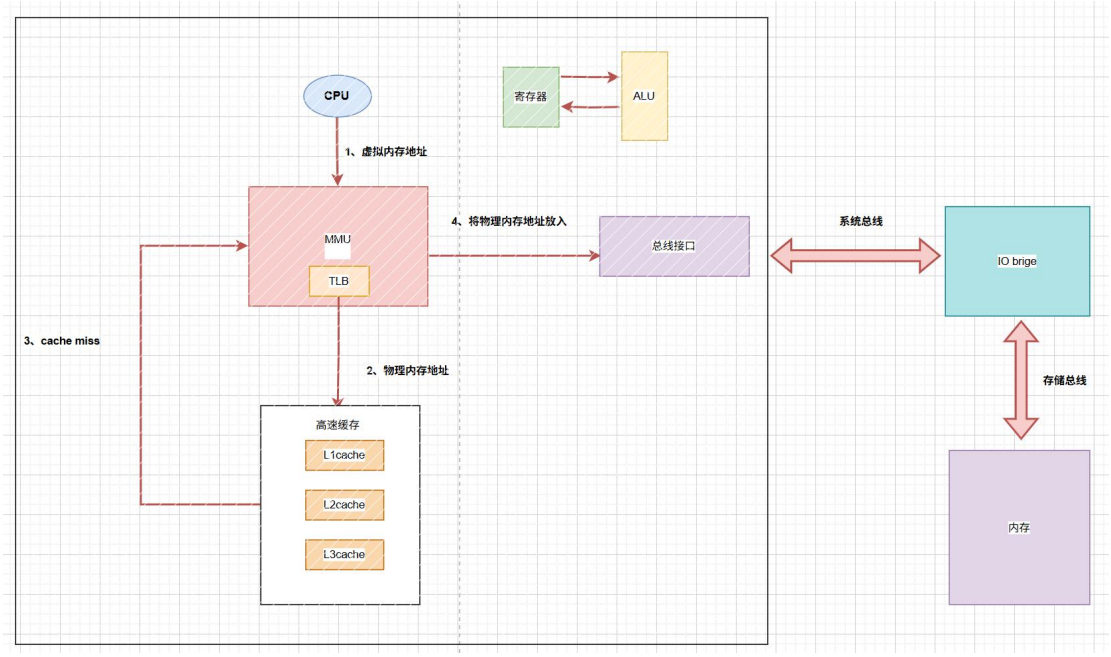
同样的道理，在 MMU 经过层层的页目录遍历之后，终于定位到了一级页表中的 PTE，MMU 也是先会从 CPU 高速缓存中去获取 PTE，如果 PTE 不在高速缓存中，MMU 才会去内存中去获取。获取到 PTE 之后，MMU 就得到了虚拟内存地址背后映射的物理内存地址。



在引入 MMU 之后，虽然加快了整个页表遍历的过程，但是 CPU 每访问一个虚拟内存地址，MMU 还是需要查找一次 PTE，即便是最好的情况，MMU 也还是需要到 CPU 高速缓存中去找一下的，即便这样开销已经很小了，但是我们还是想近一步降低这个访问 CPU CACHE 的开销，让 CPU 访存性能达到极致。

既然 MMU 每次都需要查找一次 PTE，那么我们能不能在 MMU 中引入一层硬件缓存，这样 MMU 可以把查找到的 PTE 缓存在硬件中，下次再需要的时候直接到硬件缓存中拿现成的 PTE 就可以了，这样一来，CPU 的访存效率又被近一步加快了。

这个 MMU 中的硬件缓存就叫做 TLB(Translation Lookaside Buffer)，TLB 是一个非常小的，虚拟寻址的硬件缓存，专门用来缓存被 MMU 翻译之后的热点 PTE。引入 TLB 之后，整个寻址过程又有了一些新的变化：



1、当 MMU 获取到最终的物理内存地址，首先会根据物理内存地址到 CPU 高速缓存中去查找数据，如果 cache hit，整个访存操作快速结束。

2、如果 cache miss，那么 MMU 会将物理内存地址放到系统总线上传输，随后 IO bridge 会将系统总线上传输的地址信号传递到存储总线上。

3、内存中的存储控制器感受到存储总线上的地址信号之后，会将物理内存地址从存储总线上读取出来。并根据物理内存地址定位到具体的存储器模块，随后解析物理内存地址从 DRAM 芯片中取出对应物理内存地址里的数据。

4、存储控制器将读取到的数据放到存储总线传输上，随后 IO bridge 将存储总线上的数据信号转换为系统总线上的数据信号，然后继续沿着系统总线传递。

5、CPU 芯片感受到系统总线上的数据信号之后，将数据从系统总线上读取出来并拷贝到寄存器中，随后通过 ALU 完成计算。